

TECHNICAL MANUAL

Software Documentation

Architecture, algorithms, parameters, REST API, data and operations of the whole Jarvis software.

Reference installation: Lenovo ThinkCentre M73 Tiny · Ubuntu Server 26.04 LTS · ONVIF PTZ camera · Novoferm Novomatic 200 garage door operator. 100 % on-premises processing.

DOCUMENT CONTROL

Document number	JRV-TM-001
Revision	B
Software version	Jarvis 0.9.0 (jarvis/VERSION)
Date of issue	2026-09-30
Author	Thierry Gayet <thierry.gayet@labworks.fr>
License	0BSD (Zero-Clause BSD)
Language	US English

RECORD OF REVISIONS

Rev	Date	Description
A	2026-09-29	Initial release (French edition).
B	2026-09-30	US English edition. Content aligned with the current repository (web front end with Anubis, MQTT, remote syslog, monitoring, NTP, TLS certificate lifecycle, camera power and end-of-life data). Contents with page numbers, glossary, bibliography and index added.

CONTENTS

About this document	4
1. What the system does	5
Typical scenarios	5
2. Software architecture	6
2.1 Processes and isolation	6
2.2 Python modules	6
2.3 Core process threads	8
2.4 Inter-process communication	8
2.5 Internal events (events.py)	9
2.6 End-to-end sequence: from face to door opening	9
3. Vision: person detection and tracking, PTZ	10
3.1 Acquisition (RtspCamera)	10
3.2 Person detection (PersonDetector)	10
3.3 Multi-object tracking (ByteTrack)	10
3.4 Target selection (select_target)	10
3.5 PTZ control (compute_command, PTZTracker, OnvifPTZ)	10
3.6 Annotated preview (PreviewWriter)	11
4. Face recognition and identification	12
4.1 Search region	12
4.2 Detection and encoding (FaceEngine, InsightFace buffalo_s)	12
4.3 Matching (EmbeddingGallery)	12
4.4 Identification by voting (IdentityResolver)	12
4.5 Unknown visitors: deduplication and clustering (UnknownStore)	13
4.6 Adaptive learning (AdaptiveEnroller)	13
4.7 Search by face	13
5. Sighting traceability	14
6. Voice: wake word, speech recognition, speaker, speech synthesis	15
6.1 Audio capture	15
6.2 Half-duplex	15
6.3 Wake word	15
6.4 Speech recognition: Vosk with a restricted grammar	15
6.5 Command parsing (CommandParser)	16
6.6 Speaker verification (option speaker.enabled)	16
6.7 Speech synthesis (Piper)	16
7. Decision engine and access rules	17
7.1 Event handling (DecisionEngine)	17
7.2 Per-person access rules (access.py)	17
8. Hardware: relay, indicator LEDs, door sensor	18
9. Administration	19
9.1 Access and interface security	19
9.2 Web interface tabs	19
9.3 Common procedures	20
9.4 Command line	20
10. REST API reference	22
11. Data and storage	24
11.1 SQLite schema (db.py)	24
11.2 Directory layout	24
11.3 Retention (maintenance thread, every 6 h)	24
12. Event log and notifications	25
12.1 Event types (events table, Log tab)	25
12.2 Webhook notifications (notify.py)	25

13. Configuration reference	26
14. Operations, performance, tests	29
14.1 Operations	29
14.2 Measured and estimated performance	29
14.3 Automated tests	29
15. Known limitations	31
16. Deployment and operations tooling	32
16.1 Central version file (jarvis/VERSION)	32
16.2 TLS manager (scripts/jarvis-cert.sh, installed as /usr/local/sbin/jarvis-cert)	32
16.3 Web front end: nginx and Anubis	33
16.4 Capability snapshot and SSH MOTD	33
16.5 Hardware inventory (scripts/jarvis-hwinfo.sh)	33
16.6 Shared bash library (scripts/lib/jarvis-common.sh)	33
16.7 Ansible deployment (deploy/ansible/)	34
17. License plate recognition and vehicle automation	35
17.1 Pipeline	35
17.2 Decisions (DecisionEngine._on_vehicle)	35
17.3 Registry, API and interface	36
17.4 Models and limits	36
18. Settings tabs and simulation	37
18.1 Tabbed settings	37
18.2 Camera simulation	37
18.3 Voice simulation	37
18.4 Safety: the relay	38
Appendix A Glossary	39
Appendix B Bibliography	43
B.1 Books in English	43
B.2 Books in French	43
B.3 Standards, specifications and papers	44
Appendix C Index	45

ABOUT THIS DOCUMENT

This document describes the software end to end. For each part, it explains what it does, how it does it, with which algorithms and parameters, and how it is administered.

- `docs/ARCHITECTURE.md` gives the big picture and justifies the design choices.
- `docs/diagrams/*.pdf` cover the network, the hardware and the wiring.
- `docs/sbom/` contains the software inventory.

Software version: see the central file `jarvis/VERSION` (command `jarvis version`). References of the form `file.py` point into the `jarvis/` package.

1. WHAT THE SYSTEM DOES

Jarvis is a **100 % local smart gatekeeper**. No data ever leaves the house. It does four things:

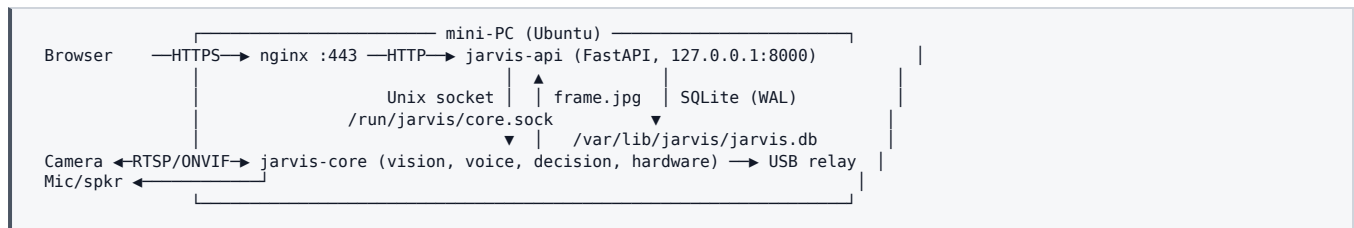
1. **It spots people and follows them** with a PTZ camera: the camera pans and zooms to keep the person in the center of the frame.
2. **It identifies faces** and records every sighting (date, time, identity or "unknown", photo). Returning unknown visitors are grouped automatically.
3. **It talks**, offline: wake word "hey jarvis", commands « ouvre / ferme le garage » ("open / close the garage"), spoken replies.
4. **It opens the garage door**, and only when every condition is met: an authorized face seen less than 60 s ago, access rules satisfied, matching voice (optional), consistent door state.

Typical scenarios

Situation	What happens
A resident arrives	The camera follows and recognizes them in under a second. The green LED lights for the recognition window (30 s by default) and « Bonjour Alice » is spoken (once a day). During the window only: « Jarvis... ouvre la porte du garage »: pulse on the relay and « J'ouvre le garage ». The LED turns off at the end of the window, and the microphone is ignored again.
A contractor comes outside their hours	They are recognized and greeted, but the authorization window stays closed. The <code>access_denied_schedule</code> event is logged and notified. Their voice command is refused.
A stranger rings	Red LED, "unknown" sighting recorded with their photo. If they have been here before, they join the "unknown #12" cluster. Webhook sent to Home Assistant.
A flagged person (watchlist)	Red LED, no greeting, <code>watchlist_seen</code> alert (log and webhook). Access is never granted.
The administrator, from the couch	Watches the annotated video stream, the history filtered by date or by person, runs a search by photo, exports to CSV, enrolls a person, sets time windows.

2. SOFTWARE ARCHITECTURE

2.1 Processes and isolation



When Anubis is deployed (§16.3), it sits between nginx and the API: nginx :443 → Anubis 127.0.0.1:8923 → API 127.0.0.1:8000.

Process	systemd unit	User	Access	Hardening
jarvis core	jarvis-core.service	jarvis (+ groups audio, dialout, plugdev)	camera (network), mic/speaker, relay, sensor, models	NoNewPrivileges, ProtectSystem=strict, ProtectHome, PrivateTmp, ReadWritePaths=/var/lib/jarvis /run/jarvis, Nice=-5, OMP_NUM_THREADS=3
jarvis api	jarvis-api.service	jarvis	no device	Same hardening, plus PrivateDevices=yes, empty CapabilityBoundingSet=, SystemCallFilter=@system-service, RestrictAddressFamilies=AF_UNIX AF_INET AF_INET6
nginx	nginx.service	www-data	ports 80/443	TLS 1.2/1.3, HSTS, strict CSP, limit_req on /api/login
Anubis (optional)	anubis@jarvis.service	dynamic user (DynamicUser=yes)	loopback only (127.0.0.1:8923)	drop-in 10-jarvis.conf: PrivateDevices=yes, empty CapabilityBoundingSet=, SystemCallFilter=@system-service, signing key passed as a systemd credential

Principle of least privilege: the API is exposed to the network but cannot actuate anything by itself. Every hardware action, and every computation that needs a model, goes through a **closed list of commands** on the core socket.

2.2 Python modules

The package is split into subpackages by concern (paths relative to `jarvis/`).

Module	Role	Main classes / functions
<code>__init__.py</code>	package root; <code>__version__</code> read from <code>jarvis/VERSION</code>	<code>__version__</code> , <code>VERSION_FILE</code>
<code>cli/__init__.py</code> , <code>cli/shell.py</code>	jarvis entry point: cmd2 administration shell or a single command	<code>main</code> , <code>JarvisShell</code>
<code>cli/tasks.py</code>	operational tasks shared by the shell and the local console	<code>run_core</code> , <code>run_api</code> , <code>CheckResult</code> , <code>BackupResult</code>
<code>console.py</code>	pfSense-style local appliance console (tty1)	<code>Console</code>
<code>config/settings.py</code>	YAML configuration and environment overrides, validated with pydantic	<code>Settings</code> and its sections, <code>load_settings</code> , <code>resolve_credentials</code>
<code>config/catalog.py</code>	catalog of the settings editable from the web UI (hot reload)	<code>Param</code> , <code>apply_overrides</code> , <code>validate_changes</code> , <code>diff</code>

Module	Role	Main classes / functions
core/service.py	the core process: assembles camera, vision, voice, decision, hardware, control socket	Core
core/control.py	API → core control channel (Unix socket, one JSON request per line)	ControlServer, ControlClient, CoreUnavailable
core/decision.py, core/access.py	GO / NO_GO decision engine; per-person access rules	DecisionEngine, access_allowed
core/events.py	events exchanged between the perception pipelines and the decision engine	FaceRecognized, UnknownFaceSeen, VoiceCommand, ManualGarage, Intent
core/logs.py	journald console, /var/log/jarvis/jarvis.log with rotation, remote syslog	configure, rotate_if_needed
core/monitoring.py	Prometheus metrics; node_exporter / Promtail / Alloy agent control	render_metrics, apply
core/notify.py	non-blocking outgoing webhook	WebhookNotifier
core/sdnotify.py	dependency-free systemd notification protocol	notify, watchdog_interval_s
core/sysinfo.py	hardware capability detection, auto-tuning, live resource metrics, capabilities.json	detect_capabilities, apply_auto_performance, ResourceSampler, write_capabilities_snapshot
vision/camera.py	RTSP frame acquisition	RtspCamera
vision/ptz.py	ONVIF PTZ control and tracking controller	OnvifPTZ, NullPTZ, PTZTracker, build_ptz
vision/pipeline.py	vision pipeline: detection, tracking, unknown faces, sightings, adaptive enrollment	PersonDetector, UnknownStore, SightingRecorder, AdaptiveEnroller, PreviewWriter
vision/faces.py	InsightFace detection (SCRFD) and embeddings (ArcFace), gallery, voting	FaceEngine, EmbeddingGallery, IdentityResolver
vision/recorder.py	event-triggered clips and time-lapse capture, FIFO quota	Recorder, enforce_quota, timelapse_frames
voice/assistant.py	voice assistant loop	VoiceAssistant
voice/wakeword.py	wake word: openWakeWord or Porcupine	OpenWakeWord, PorcupineWakeWord, build_wakeword
voice/commands.py	pure command parsing	CommandParser, normalize
voice/speaker.py, voice/tts.py, voice/audio.py	ECAPA speaker verification; Piper speech synthesis; audio helpers	SpeakerVerifier, PiperTTS, Framer
hardware/devices.py	outputs (garage relay, LEDs) and door sensor input	SerialLcusOutput, HidDcttechOutput, GpiodOutput, MockOutput, DoorSensor
integrations/mqtt.py	MQTT bridge, Home Assistant discovery	MqttBridge, discovery_messages
storage/database.py	SQLite persistence (WAL), migrations, audit hash chain	Database, Person, event_hash
provisioning/models.py	model download and export	setup_yolo, setup_insightface, setup_vosk, setup_piper, setup_openwakeword, setup_speechbrain
web/app.py	REST API (FastAPI) behind nginx and Anubis	create_app, LoginRateLimiter
web/static/	framework-free UI: index.html, app.js, style.css, i18n/*.json (12 locales), img/	—

2.3 Core process threads

The core is **multi-threaded** rather than multi-process. The heavy computations (FFmpeg, OpenVINO, ONNX Runtime, Kaldi) run in native code and release the GIL. Each library manages its own thread pool, and the models are loaded into memory only once.

Thread	Class	Rate	Inputs → outputs
camera	RtspCamera	stream rate	RTSP → latest frame + sequence number (condition variable)
vision	VisionPipeline	vision.process_fps (6/s)	frame → tracks, PTZ commands, face events, sightings, frame.jpg
ptz-onvif	OnvifPTZ (asyncio loop)	on demand	latest pending command → ONVIF SOAP
voice	VoiceAssistant	80 ms blocks	microphone → VoiceCommand
tts	PiperTTS	queue	text → cached WAV → speaker
decision	DecisionEngine	event queue	events → LEDs, relay, voice, log, notifications
notify	WebhookNotifier	queue (100 max)	event → HTTP POST (if webhook_url)
control	ControlServer	on demand	JSON over socket → handler → JSON
maintenance	Core._maintenance	every 6 h	retention purge (GDPR)
LED timer	threading.Timer	one-shot	turns the indicator LEDs off after led_on_s

Graceful shutdown (`Core.shutdown`, triggered by SIGTERM or SIGINT):

1. components are stopped;
2. `join(5 s)` on `vision`, `camera` and `voice`: the core never exits in the middle of a native inference (that case used to cause a `terminate called...`);
3. ONVIF is closed;
4. the relay is returned to rest;
5. `core_stopped` event.

2.4 Inter-process communication

Control socket (`control.py`): `/run/jarvis/core.sock`, mode `0660` (`umask 007`), group `jarvis`. Every request and every response fits on **one JSON line**. The server is a `ThreadingUnixStreamServer`, and the client has a 30 s timeout.

Command	Parameters	Response	Used by
status	—	camera_connected, vision_fps, tracks, target_id, known_embeddings, voice_state, door, authorized_persons	Live tab
reload_faces	—	known_embeddings	after any change to faces or persons
reload_voices	—	—	after a change to voice profiles
enroll_face	person_id, image_path	face_id, det_score	adding a photo
embed_face	image_path	embedding[512], det_score	search by face (nothing is stored)
enroll_voice	person_id, audio_path	profile_id	adding a voice
garage_pulse	actor	—	"Garage pulse" button
ptz_home	—	—	"PTZ: home position" button

Command	Parameters	Response	Used by
say	text (200 characters max)	—	"Make Jarvis speak" field

Every response contains `ok: true|false`, plus `error` on failure. An unknown command is rejected.

Preview frame: `/run/jarvis/frame.jpg` on `tmpfs`. The core writes it to `frame.tmp`, then replaces it atomically (`os.replace`). The API reads it back at `preview_fps` and streams it as MJPEG. The RTSP stream is therefore **decoded only once**.

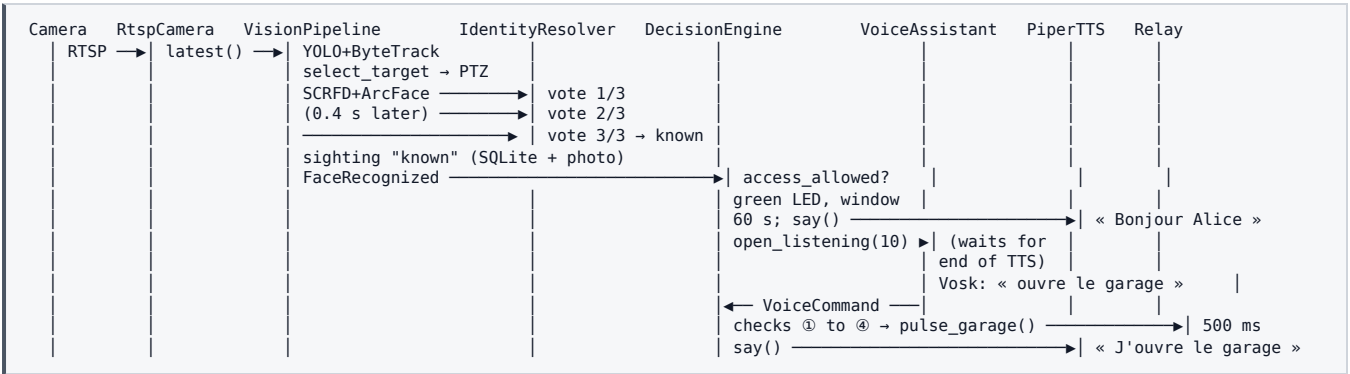
SQLite in WAL mode: shared by both processes. One connection is opened per operation, with `busy_timeout=5000` and `foreign_keys=0N`. WAL mode allows concurrent reads while one process is writing.

2.5 Internal events (events.py)

```
FaceRecognized(track_id, person_id, first_name, score, ts, sighting_id)
UnknownFaceSeen(track_id, unknown_id|None, ts, cluster_id, sighting_id)
VoiceCommand(text, intent|None, speaker_person_id, speaker_score, via_wakeword, ts)
ManualGarage(actor, ts)
Intent = OPEN_GARAGE | CLOSE_GARAGE | CANCEL
```

The vision and voice pipelines **produce** these events into a single `queue.Queue`. The `DecisionEngine` is the **only consumer**, which serializes all decisions.

2.6 End-to-end sequence: from face to door opening



3. VISION: PERSON DETECTION AND TRACKING, PTZ

3.1 Acquisition (RtspCamera)

- OpenCV with the FFMpeg backend. Options `rtsp_transport=tcp` and `stimeout=5 s` (variable `OPENCV_FFMPEG_CAPTURE_OPTIONS`), `CAP_PROP_BUFFERSIZE=1`.
- The thread reads **continuously** and keeps only **the latest frame**, with a sequence number. Consumers call `latest(after_seq)`, which waits on a `threading.Condition` until a newer frame arrives. FFMpeg buffer latency therefore never accumulates.
- Reconnection**: if opening fails, the delay doubles at each attempt (from `reconnect_delay_s = 2 s` up to 30 s). After a disconnection, a 1 s pause prevents a tight loop.
- `hw_accel: true` requests VAAPL decoding (`VIDEO_ACCELERATION_ANY`). On the Intel HD 4600, this only works with H.264.
- Use the **720p H.264 substream**: decoding 4K H.265 would saturate a Haswell CPU.

3.2 Person detection (PersonDetector)

- Ultralytics **YOLO11n** model, exported to **OpenVINO IR FP32** by `jarvis setup-models (yolo11n.pt → yolo11n_openvino_model/)`.
- Call: `model.track(frame, persist=True, classes=[0], conf=0.45, imsz=480, tracker="bytetrack.yaml")`. COCO class 0 is "person". The frame is *letterboxed* to 480 px on the long side. Detections below `conf` are discarded.
- Output: a list of `Track(id, box xyxy, conf)` in source-frame pixels.

3.3 Multi-object tracking (ByteTrack)

ByteTrack is built into Ultralytics.

- A **Kalman filter** predicts the position of each track.
- Association is done **in two passes** by IoU: first the high-confidence detections, then the low-confidence ones (often partially occluded people), matched to the tracks left orphaned.
- There is no re-identification network to run. This is the right option for a single camera on a modest CPU.
- On the Jarvis side, a track absent for more than `TRACK_GRACE_S = 5 s` is forgotten, and the voting state of the `IdentityResolver` is purged with it (`prune`).

3.4 Target selection (select_target)

- If the previous target is still present, it is kept. This stability prevents the camera from swinging back and forth between two people.
- Otherwise, the best score $0.6 \times \min(4 \times \text{relative area}, 1) + 0.4 \times \text{centrality}$ wins, with `centrality = 1 - \min(2 \times |cx - 0.5|, 1)`. In other words: the closest and most central person.

3.5 PTZ control (compute_command, PTZTracker, OnvifPTZ)

Proportional controller, speeds normalized to `[-1, 1]`:

```
ex = cx / W * 2 - 1                                (-1 left ... +1 right)
ey = (y1 + 0.3 * h) / H * 2 - 1                    (aim at 30 % from the top of the box: the face)
pan = 0 if |ex| < deadzone, else clamp(pan_gain * ex, max_speed)
tilt = 0 if |ey| < deadzone, else clamp(-tilt_gain * ey, max_speed)  (ONVIF: y+ = up)
zoom: ratio = h / H ; err = target_height_ratio - ratio
      box touching an edge and ratio > 0.3          → zoom = -zoom_gain * 0.5 (safety zoom-out)
      |err| > 0.1 and (err < 0 or target centered) → zoom = clamp(zoom_gain * err, max_speed)
```

- Dead zone** of 12 %, **speed capped** at 0.5 and **low gains**: RTSP latency (0.3 to 1 s) would make a more aggressive controller oscillate.
- Zooming in only happens when the target is **centered**.
- Anti-flooding**: a command is sent only if it changes, and at most every `command_interval_s` (0.25 s). A null command sends a `Stop`.

- **Return to preset** `home_preset` after `return_home_after_s` (30 s) without a target.
- **Sound trigger** (optional): `sound_trigger_rms > 0` points the camera at `sound_preset` on a noise, if no target has been tracked for 5 s.
- **ONVIF**: `onvif-zeep-async` runs in its own `asyncio` loop (thread `ptz-onvif`). Commands are **coalesced**: only the most recent one is sent, and vision never waits on the network. Calls used: `ContinuousMove`, `Stop` and `GotoPreset` on profile `profile_index`. On connection failure, retry with a delay growing from 5 s to 120 s.
- **Fixed camera** (`ptz.enabled: false`): tracking is disabled (`PTZTracker.enabled = False`) and the camera is never "moving". Before this fix, face recognition never ran without PTZ.

3.6 Annotated preview (PreviewWriter)

Rate `preview_fps` (5/s). The frame is annotated with:

- green boxes for known people (with their first name), red for unknown ones, yellow for people not yet identified;
- a thick outline for the target;
- track number and clock.

It is then resized to `preview_width` (960 px), encoded as JPEG quality 75 and written atomically.

4. FACE RECOGNITION AND IDENTIFICATION

4.1 Search region

The face is searched only in the **upper half of the "person" box** (`face_region`), widened by 10 % on the sides and 5 % upward, over 55 % of the height. This is far cheaper than analyzing the whole frame, and the face is attached to the track without ambiguity. A region smaller than 48 px is ignored.

Budget: at most `max_faces_per_frame` (2) faces per frame, starting with the target. Each track is rechecked at most every `recheck_interval_s` (0.4 s). **Nothing is analyzed during a PTZ movement**, because the frame is blurred.

4.2 Detection and encoding (FaceEngine, InsightFace buffalo_s)

- **SCRFD-500M** detects faces and their 5 landmarks, with a `det_size` input of 320 × 320, on ONNX Runtime CPU.
- **Alignment:** similarity transform to the 112 × 112 ArcFace template, from the 5 landmarks.
- **ArcFace MobileFaceNet** produces a **512-dimensional embedding**, L2-normalized (`normed_embedding`).
- **Quality filters** (`quality_ok`): shortest side $\geq$ `min_face_px` (40 px), detection score $\geq$ `min_det_score` (0.6), sharpness $\geq$ `min_sharpness` (30). Sharpness is the **variance of the Laplacian** in grayscale.
- `best_face` keeps the largest face that passes the filters. For enrollment from a photo, the image is first downscaled to 960 px and the filters are lifted.
- A lock (`threading.Lock`) serializes access to the ONNX session between the vision thread and the control commands.

4.3 Matching (EmbeddingGallery)

- The gallery is an in-memory $N \times 512$ matrix (float32, normalized rows) along with the vector of `person_ids`. A person can have several photos, hence several rows.
- **Cosine similarity** is a plain dot product, `sims = M · e`. The best candidate is the `argmax`. The computation is instantaneous up to thousands of embeddings.
- ArcFace threshold `match_threshold`:

Threshold	Behavior
0.40	permissive: more recognitions, and more confusions
0.45	default: a good compromise for a household
0.55	strict: more "not recognized" in profile view or under IR

- The gallery is **hot-reloaded** (`reload_faces`) after any addition or deletion, without restarting the core.

4.4 Identification by voting (IdentityResolver)

A single blurred frame must **never** trigger an action. The resolver therefore keeps a per-track state:

```
for each observation (track t, best candidate p, score s, embedding e, capture c, quality q):
    if t is already decided: ignore
    if s  $\geq$  match_threshold:
        votes[t][p] += 1 ; remember the best observation (s, q, e, c) for p
        if votes[t][p]  $\geq$  votes_required (3): decide KNOWN(p) with the best observation
    else:
        misses[t] += 1 ; remember the highest-quality observation
        if misses[t]  $\geq$  unknown_after_observations (6) and no vote: decide UNKNOWN
```

- The decision is **made once per track**. A person standing still in front of the camera therefore generates a single sighting.
- Quality of a capture: `q = det_score × min(width / 100 px, 1) × min(sharpness / 100, 1)`, between 0 and 1.
- After a gallery reload, all tracks are re-evaluated (`reset`).

4.5 Unknown visitors: deduplication and clustering (UnknownStore)

When a track is declared unknown, its **best capture** is processed as follows:

1. **Nearest-neighbor search** among all stored unknowns (in-memory cache reloaded from SQLite).
2. **Deduplication**: if the similarity is $\geq$ `unknown_dedupe_similarity` (0.5) with an unknown seen less than `unknown_dedupe_window_s` (1 h) ago, nothing is stored. It is the same visit. The sighting itself is still recorded and attached to the cluster.
3. **Clustering** (incremental *leader* clustering): if the similarity is $\geq$ `unknown_cluster_similarity` (0.5), the new unknown **joins the cluster** of its nearest neighbor. Otherwise, it founds a new cluster (`cluster_id = id`).
4. The capture is stored in `unknown/YYYYMMDD/<uuid>.jpg`, with its embedding.

Labeling a cluster (Unknowns tab) turns all of its visits at once into known faces of the chosen or newly created person. The embeddings are reused without recomputation. The cluster is also **identified retroactively**: its sightings switch to the `labeled` status, with the `person_id` (\$5).

4.6 Adaptive learning (AdaptiveEnroller)

Appearance changes: haircut, beard, glasses, aging, lighting. When a person is recognized **with certainty**, their best capture enriches their gallery, provided that **all** of the following conditions are met:

Condition	Parameter	Default	Reason
Feature enabled	<code>adaptive_enabled</code>	<code>true</code>	—
Score well above the threshold	<code>adaptive_min_score</code>	0.60 (threshold 0.45)	avoid learning a confusion
Good-quality capture	<code>adaptive_min_quality</code>	0.35	no blur, no tiny face
Genuinely new capture	<code>adaptive_novelty_max_similarity</code>	0.85	no point stacking duplicates
Per-person cap	<code>adaptive_max_per_person</code>	5	limits drift and poisoning
Not on the watchlist	—	—	—

The addition carries `source = "auto"`. It is marked "auto" in the interface, removable in one click, and logged (`face_auto_enrolled`). The gallery is reloaded immediately. **Residual risk**: a confusion with a score $\geq$ 0.6 would be learned. This is unlikely with ArcFace, where different people generally stay below 0.3. For strict control, disable the feature.

4.7 Search by face

POST `/api/search/face`:

1. The uploaded photo is re-encoded, oriented according to its EXIF data, then written to `data/tmp/`.
2. The core computes its embedding (`embed_face`), and the file is **deleted immediately**.
3. The API compares the embedding with those of **all stored sightings** and returns those above `search_threshold` (0.45), sorted by similarity.

Use case: "When did this person come by?", from a photo of a neighbor, a delivery driver or a screenshot. The search is logged (`face_search`) with its author.

5. SIGHTING TRACEABILITY

Every identification decision becomes a row of the `sightings` table:

Field	Content
<code>ts</code>	date and time (Unix timestamp, displayed in local time)
<code>status</code>	known (recognized) · unknown · watchlist (watched person) · labeled (unknown identified afterward)
<code>person_id</code>	identity (empty for an unknown)
<code>score</code>	ArcFace similarity of the best observation
<code>quality</code>	capture quality (\$4.4)
<code>image_path</code>	face photo (<code>sightings/YYYYMMDD/<uuid>.jpg</code> , JPEG quality 85)
<code>embedding</code>	512 floats (used by the search by face)
<code>track_id</code>	ByteTrack track number
<code>unknown_id,</code> <code>cluster_id</code>	stored unknown and visitor cluster

Browsing: `Sightings` tab.

- Filters by period (from / to), person and status, with pagination.
- Table: date, time, thumbnail, identity, status, score, track.
- 14-day histogram (recognized in green, unknown or watched in red).
- **CSV export:** `;` separator, UTF-8 BOM for Excel, columns `id;date;time;status;person;score;quality;track;unknown_group`. Every export is logged.

Retention: `sighting_retention_days` (90 days), photos included. Deleting a person also erases all of their sightings (right to erasure).

6. VOICE: WAKE WORD, SPEECH RECOGNITION, SPEAKER, SPEECH SYNTHESIS

6.1 Audio capture

- `sounddevice.RawInputStream`: 16 kHz, mono, int16, blocks of **1,280 samples (80 ms)**, the native size of `openWakeWord`.
- The `PortAudio` callback pushes blocks into a queue of 200 blocks (16 s). When it is full, **audio is dropped rather than accumulated**: better to lose a bit of sound than to add latency.
- On a device error, the stream is restarted after 5 s.

6.2 Half-duplex

- The `PiperTTS.speaking` flag is raised **as soon as a sentence is queued**, then lowered 0.3 s after playback ends (reverberation tail).
- While it is raised, the microphone is ignored and the wake-word state is reset. Jarvis therefore never hears itself.
- The `ReSpeaker` adds its hardware echo cancellation.

6.3 Wake word

- **`openWakeWord`** (default, "hey jarvis") is an ONNX chain of three models: 1. mel spectrogram; 2. Google's audio embedding extractor (pre-trained, frozen); 3. small `hey_jarvis_v0.1` classifier.

A score $\geq$ `threshold` (0.5) triggers wake-up. `Framer` re-slices the stream into frames of the size required by the engine. - **Porcupine** (optional, "jarvis" alone) requires a Picovoice key. Sensitivity `porcupine_sensitivity`. - After detection: 880 Hz beep (150 ms, Hann window), audio queue flush, then listening for `listen_timeout_s` (6 s).

Recognition window (`decision.auth_window_s`, 30 s by default, set in Settings > Decision and access):

- **Opening**: when an authorized person is recognized, the decision engine lights the green LED for the whole window and calls `VoiceAssistant.activate(window)`. The event is `voice_window_opened`, with the person and the duration.
- **Standby**: outside the window, with `decision.voice_only_after_recognition` (default), the microphone is **ignored**: no wake word detection at all. The status shows `voice_state = sleeping`.
- **Extension**: a new recognition extends the window (it never shortens it); the LED timer restarts.
- **Expiry**: the LED turns off, the assistant goes back to sleep, and the service log records "Voice commands disabled: recognition window closed".
- **Wake word required**: every command must start with the **"Jarvis" token**, i.e. the wake word ("hey jarvis" with `openWakeWord`, "jarvis" with `Porcupine`). Listening without it after the greeting (`listen_after_recognition_s`) is 0, i.e. disabled.

6.4 Speech recognition: Vosk with a restricted grammar

- Model `vosk-model-small-fr-0.22` (Kaldi, about 41 MB), `KaldiRecognizer(model, 16000, grammar)`.
- The **grammar** is the JSON list of the configured phrases, normalized, plus "[unk]". By default (`commands.strict`) it holds exactly **two** phrases: « ouvre la porte du garage » and « ferme la porte du garage ». The decoder can produce **only** these phrases. Everything else comes out as [unk]. This is what makes recognition very robust to noise, far more than free dictation.
- Loop: every block goes through `AcceptWaveform`. At each end of utterance (`Result`):
- phrase understood: stop;
- phrase not understood after the wake word: stop as well, and Jarvis replies « Je n'ai pas compris » ("I did not understand");
- noise during direct listening: keep listening.
- When the time limit is reached: `FinalResult`.

6.5 Command parsing (CommandParser)

1. **Normalization**: lowercase, NFKD decomposition, removal of accents and apostrophes, collapsed whitespace (« Fermé l'accès » → « ferme l'accès »).
2. **Exact match** against the configured phrases.
3. **Strict mode** (`commands.strict`, default): nothing else is accepted. « ouvre le garage » or « ferme la porte » are refused, as is anything that is not one of the two commands.
4. **Keyword fallback** (only with `commands.strict: false`): presence of `garage` or `porte` ("door"), plus a word starting with `ouvr` (*ouvrir*, open) or `ferm` (*fermer*, close).
5. Otherwise: None, command not understood.

6.6 Speaker verification (option `speaker.enabled`)

- **SpeechBrain ECAPA-TDNN** (`spkrec-ecapa-voxceleb`): normalized 192-dimensional embedding, on CPU.
- **Enrollment**: 10 s read aloud in the browser (`MediaRecorder`, `WebM/Opus`). The recording is converted by FFmpeg to float32 16 kHz mono (3 s minimum), then stored in `voice_profiles`.
- **Identification**: the command utterance (`max_utterance_s` = 6 s at most, 0.5 s at least) is compared with the voice gallery. Below `threshold` (0.35), the speaker is not identified. A short command (1 to 2 s) yields a less reliable voiceprint, hence this compromise threshold.
- Verification **does not detect a replayed** recording. It complements the face, it does not replace it.

6.7 Speech synthesis (Piper)

- **Piper** (VITS network exported to ONNX), voice `fr_FR-siwis-medium`, 22.05 kHz, offline. Compatible with the `piper-tts` 1.2 API (`synthesize`) and 1.3 onward (`synthesize_wav`).
- **Disk cache**: key `SHA-1(model | text)` → `tts-cache/<key>.wav`. Each sentence is synthesized only once, and greetings become instantaneous.
- Playback through `sounddevice.play` on `audio.output_device`. FIFO queue: sentences never overlap.

7. DECISION ENGINE AND ACCESS RULES

7.1 Event handling (DecisionEngine)

Event	Actions
FaceRecognized, person on the watchlist	Red LED for led_on_s, watchlist_seen event, notification, optional watchlist_message. No greeting and no authorization.
FaceRecognized, regular person	face_recognized event; green LED for the whole recognition window. If can_open_garage and access_allowed(person, now): the person enters the recognition window (auth_window_s = 30 s) and the voice commands are enabled for it (voice_window_opened). Otherwise, access_denied_schedule with the reason, and notification. Greeting once a day (greetings table).
UnknownFaceSeen	Red LED, face_unknown event (unknown, cluster, sighting), notification, optional unknown_message.
VoiceCommand without intent	« Je n'ai pas compris. » (voice_not_understood)
VoiceCommand CANCEL	« D'accord. » (voice_cancel)
VoiceCommand OPEN / CLOSE	Checks ① to ④ below, then pulse.
ManualGarage (web)	Direct pulse, logged with the user.

Checks before a voice-triggered pulse:

1. at least one authorized person in the recognition window (30 s by default), otherwise « Accès refusé. Je ne vous ai pas reconnu. » ("Access denied. I did not recognize you.", no_authorized_face);
2. if speaker.enabled: the identified voice must belong to a person in the window, otherwise « Voix non reconnue » ("Voice not recognized", speaker_mismatch). Without voice verification, the command is attributed to the most recently recognized person;
3. door sensor: "open" on a door that is already open yields « Le garage est déjà ouvert. » ("The garage is already open."), and vice versa;
4. hardware debounce: cooldown_s (5 s) minimum between two pulses, otherwise garage_pulse_rejected.

Every refusal lights the red LED, plays a voice message and creates the voice_denied event with its reason, followed by a notification.

7.2 Per-person access rules (access.py)

Field	Format	Effect
can_open_garage	boolean	base condition
watchlist	boolean	forbids any access (absolute priority)
valid_until	timestamp	access expires after this date (guest, contractor, rental)
access_days	ISO digits 1 to 7 (Monday = 1)	allowed days. Empty = every day
access_start, access_end	HH:MM	time range [start, end[, which may cross midnight (22:00 → 06:00)

access_allowed(person, date) returns (allowed, reason). Possible reasons: ok, not_authorized, watchlist, expired, day_not_allowed, outside_hours. The function is pure, tested by 12 edge cases (tests/test_features.py).

8. HARDWARE: RELAY, INDICATOR LEDS, DOOR SENSOR

Backend (<code>hardware.*.backend</code>)	Hardware	Protocol
<code>serial_lcus</code>	CH340 USB relay board "LCUS-1/2/4"	Serial 9,600 baud, frame A0 <channel> <state> <checksum> (checksum = (A0 + channel + state) & FF). A single port shared by all channels (<code>_SharedSerial</code>), protected by a lock.
<code>hid_dcttech</code>	"USBRelay" USB HID relay (16c0:05df)	<i>Feature report</i> [0, FF (on) or FD (off), channel, 0...]
<code>gpiod</code>	Linux GPIO (libgpiod v2): Raspberry Pi, FT232H, MCP2221	output line, optional <code>active_low</code>
<code>mock</code>	none	logs only (development, automatic fallback when opening fails)

- **Garage pulse** (`Hardware.pulse_garage`): the contact is closed for `pulse_ms` (500 ms), then opening is **guaranteed** by a `finally`. Lock and `cooldown_s`. The return value is `False` when the pulse is refused.
- **Indicator LEDs** (`indicate`): turns one color on and the other off. A timer turns them off after `led_on_s`, and a new indication cancels the previous one.
- **Door sensor:**
 - `serial_cts` reads the CTS pin of a USB-serial adapter. With an FT232RL TTL adapter, the reed switch is wired between CTS and GND. With an RS-232 port, between DTR and CTS;
 - `gpiod` reads a GPIO line with pull-up;
 - `active_means_closed` sets the logic (true = contact closed ↔ door closed).
- **Shutdown:** all outputs return to rest.
- Stable device names via udev (`deploy/udev/99-jarvis.rules`): `/dev/jarvis-relay` (1a86:7523) and `/dev/jarvis-door` (0403:6001).

9. ADMINISTRATION

9.1 Access and interface security

- **Accounts:** `jarvis create-user <name>` (12 characters minimum). Password hashed with **Argon2id**, with automatic rehashing when the parameters change.
- **Default account:** when the database holds no account, the API creates the factory account `admin / admin` at startup (`Database.ensure_default_admin`, event `default_admin_created`) with a **forced password change**. As long as `must_change_password` is set, every route returns 403 "Password change required" except those of `PASSWORD_CHANGE_ALLOWED` (`/api/me`, `/api/logout`, `/api/account/password`). `POST /api/account/password` checks the current password (401 otherwise) and enforces the policy: at least 12 characters, at least 5 different characters, different from the current password, and not the account name nor `admin`, `password` or `jarvis` (422 otherwise). The account's other sessions are revoked and the change is audited (`password_changed`, `password_change_failed`). `jarvis reset-admin [--yes]` restores `admin/admin` with the forced change and closes its web sessions (`admin_factory_reset`).
- **Password dialog (web UI):**
- **Forced mode:** after a sign-in with the initial password, the application shell stays visible but locked (blurred) behind a modal dialog. Escape is ignored and the secondary button signs out; no page is loaded and no polling starts until the change succeeds.
- **Any 403 "Password change required":** the dialog reopens.
- **Voluntary mode:** the user menu has a "Change password" entry.
- **Live checklist:** it mirrors the server policy (12 characters, 5 distinct characters, no default word, different from the current password, confirmation), and the submit button is enabled only when every rule is met. The server stays the authority: its 401/422 messages are shown.
- **After the change:** a toast reports the other sessions that were signed out.
- **Boot robustness (web UI):** the page never stays blank.
- **Missing translation table:** the English texts of `index.html` are used.
- **Page opened as a local file (file://):** the login view explains how to reach the appliance and disables the form.
- **API not answering:** the login view shows "service not answering".
- **First visit (401 on /api/me):** the login view appears without any "session expired" notice.
- **Localization:** `static/i18n/en-US.json` is the single reference (every key). The 11 other locales (`fr-FR`, `es-ES`, `nl-NL`, `de-DE`, `it-IT`, `ru-RU`, `zh-CN`, `id-ID`, `ko-KR`, `ja-JP`, `th-TH`) must have exactly the same keys and placeholders, which `tests/test_web_ui.py` checks.
- **Images** (`static/img/`): `favicon` (SVG, ICO), Apple touch icon, PWA icons (`manifest.webmanifest`), `login-bg.svg` (login hero background), standalone copies of the icon sprite and of the hero illustration. The page also sends `<meta name="robots" content="noindex...">`.
- **Session:** random 256-bit token (`secrets.token_urlsafe(32)`). The database stores only its **SHA-256**. Cookie `jarvis_session` with `HttpOnly`, `Secure`, `SameSite=Strict`, `lifetime session_hours` (12 h).
- **Anti-CSRF:** the `X-Jarvis: 1` header is mandatory on every request other than GET or HEAD. A third-party form cannot add it.
- **Anti-brute-force:** at most 5 failures per IP over 5 minutes (`LoginRateLimiter`), plus `nginx limit_req` (10 per minute, burst of 5). The IP is read from the `X-Real-IP` header supplied by `nginx`.
- **Media:** photos are served only by `/api/media`, which requires a session, accepts only the `faces/`, `unknown/` and `sightings/` directories (after symlink resolution) and sends `Cache-Control: private`.
- Every administrative action is **logged with its author**.

9.2 Web interface tabs

Tab	Features
Live	Annotated MJPEG stream (stopped when leaving the tab), core status refreshed every 3 s (camera, frames analyzed per second, tracks, enrolled faces, voice, door, people in the authorization window). Buttons: garage pulse (with confirmation), PTZ to preset, make Jarvis speak.

Tab	Features
Sightings	Filterable timestamped history, 14-day histogram, CSV export, search by face .
People	Creation, right to open the garage, watchlist , access rules (days, hours, expiration), photos (multiple upload, deletion, "auto" mark), voice profile (10 s recording, removal), last sighting, full deletion.
Unknowns	Clustered visitors with their number of visits and sightings, and the dates. Association with an existing or new person (retroactive identification), deletion.
Log	All events (§12), pagination into the past, colors for openings and refusals.
Vehicles	Plate registry and read history (§17.3).
Logs	Service log with text search, level filter and live tail (secrets masked).
Settings	Every hot-reloadable parameter, in domain tabs with one sub-tab per group ; camera stream test and annotated preview; simulation of the camera and the voice (§18).

9.3 Common procedures

Add a resident

1. People tab: first name, last name, check "Can open the garage", then Add.
2. **+ Photos**: 3 to 5 sharp photos, face frontal and three-quarter, with and without glasses, in daylight. Photos are re-encoded on import (EXIF and GPS stripped), downscaled to 1,600 px, then encoded by the core. A photo without a detected face is rejected.
3. Optionally, **+ Voice**: read the suggested sentence for 10 s, in a quiet place (HTTPS is required for the microphone).
4. In the Live tab, check that the person's box shows their first name.

Temporary access (contractor, guest): create the person and check "Can open the garage". Under "Access rules", choose the days and the time range (for example Mon to Fri, 08:00 to 18:00), and an "Until" date, then Save. Outside the range, the person is recognized and greeted, but is not granted access.

Revoke access: uncheck "Can open the garage"; this takes effect immediately for subsequent recognitions. An authorization window that is already open expires after 60 s at most.

Flag a person: check "Watchlist". A photo is needed to recognize them. The simplest way is to label their cluster in the Unknowns tab.

Identify a visitor afterward: in the Unknowns tab, on the cluster card, choose "Associate with..." a person or "+ New person...". All of their visits become photos of that person, and their past sightings are attributed.

Find a person's visits: Sightings tab, "person" filter. If they are not enrolled: **Search by face** with a photo.

Delete a person (right to erasure): Delete button. The following are erased: the record, the face embeddings and photos, the voice profiles and their recordings, the **sightings and their photos**, the greetings. The core reloads its galleries. The log keeps the `person_deleted` event, without any biometric data.

Export the history: Sightings tab, apply filters, then Export CSV.

9.4 Command line

Command	Role
<code>jarvis core / jarvis api</code>	services (started by systemd)
<code>jarvis setup-models</code>	downloads Vosk, Piper, YOLO11n (and its OpenVINO export), InsightFace, openWakeWord and ECAPA if enabled. This is the only step that needs Internet access.
<code>jarvis create-user <name></code>	creates or resets an account (password asked twice)
<code>jarvis reset-admin [--yes]</code>	factory reset of the admin web account (§9.1)

Command	Role
<code>jarvis enroll-face --first-name X [--last-name Y] [--can-open-garage] photo...</code>	offline enrollment, bypassing the core (initial installation)
<code>jarvis say "text"</code>	tests speech synthesis and the speaker
<code>jarvis test-hardware [--pulse]</code>	reads the sensor, lights each LED for 2 s; <code>--pulse</code> also actuates the garage
<code>jarvis version</code>	prints the software version (<code>jarvis/VERSION</code>), Python and platform
<code>option -c file.yaml</code>	alternative configuration (otherwise <code>\$JARVIS_CONFIG</code> , then <code>/etc/jarvis/config.yaml</code> , then <code>config/config.yaml</code>)

10. REST API REFERENCE

Prefix `/api`. Every route requires a session, except `POST /login`. Every method other than `GET` requires the `X-Jarvis: 1` header. Responses are JSON, except CSV, MJPEG and media. The OpenAPI schema is deliberately disabled.

Method	Route	Body / parameters	Response
POST	<code>/login</code>	<code>{username, password}</code>	<code>{username}</code> , session cookie · 401 · 429
POST	<code>/logout</code>	—	<code>{}</code>
GET	<code>/me</code>	—	<code>{username, idle_timeout_minutes, language, must_change_password, version}</code> (the version is disclosed only to authenticated users)
POST	<code>/account/password</code>	<code>{current_password, new_password}</code>	<code>{other_sessions_revoked}</code> · 401 wrong current password · 422 policy not met
GET	<code>/persons</code>	—	list: <code>id</code> , first name, last name, <code>can_open_garage</code> , <code>watchlist</code> , access rules, <code>face_count</code> , <code>auto_face_count</code> , <code>voice_count</code> , <code>last_seen</code>
POST	<code>/persons</code>	<code>{first_name, last_name?, can_open_garage?, watchlist?, access_days?, access_start?, access_end?, valid_until?}</code>	<code>{id}</code> (201) · 422 on invalid format
PATCH	<code>/persons/{id}</code>	same fields, optional, plus <code>clear_valid_until</code>	<code>{}</code> and core reload
DELETE	<code>/persons/{id}</code>	—	<code>{}</code> (full erasure)
GET	<code>/persons/{id}/faces</code>	—	photos: <code>id</code> , <code>image_path</code> , <code>source</code> (upload, cli, labeled, auto)
POST	<code>/persons/{id}/faces</code>	multipart file (image, 15 MB max)	<code>{face_id}</code> (201) · 415 · 422 if no face · 503 if the core is down
DELETE	<code>/faces/{id}</code>	—	<code>{}</code>
POST	<code>/persons/{id}/voice</code>	multipart file (WAV, WebM, OGG, MP3, M4A, FLAC)	<code>{profile_id}</code> (201)
DELETE	<code>/persons/{id}/voice</code>	—	<code>{}</code>
GET	<code>/unknowns</code>	—	individual unknowns
GET	<code>/unknowns/clusters</code>	—	clusters: <code>cluster_id</code> , faces, sightings, <code>first_seen</code> , <code>last_seen</code> , <code>image_path</code>
POST	<code>/unknowns/{id}/label</code>	<code>{person_id}</code> or <code>{new_person:{...}}</code>	<code>{person_id, face_id, sightings_relabeled}</code>
POST	<code>/unknowns/clusters/{cid}/label</code>	same	<code>{person_id, faces, sightings_relabeled}</code>
DELETE	<code>/unknowns/{id}</code> · <code>/unknowns/clusters/{cid}</code>	—	<code>{}</code> · <code>{deleted}</code>
GET	<code>/sightings</code>	<code>start</code> , <code>end</code> (YYYY-MM-DD, local time, inclusive bounds), <code>person_id</code> , <code>status</code> , <code>limit</code> (1,000 max), <code>before_id</code>	sightings with first and last name
GET	<code>/sightings.csv</code>	same filters	CSV file as an attachment

Method	Route	Body / parameters	Response
GET	/sightings/stats	days (366 max)	[{day, status, n}]
POST	/search/face	multipart file, limit (500 max)	sightings with similarity, sorted
GET	/media	path	image (allowed directories only)
GET	/events	limit, type, before_id	events
GET	/status	—	core status (see status, §2.4)
POST	/garage/pulse · /ptz/home · /say	— · — · {text}	{}
GET	/stream.mjpg	—	multipart/x-mixed-replace, 5 frames/s
GET	/settings	—	{groups: [{name, domain, params}], domains} (§18.1)
GET	/simulation	—	{files: [{name, kind, size}], camera_simulation, max_upload_mb}
POST	/simulation/files	multipart file (video, photo or recording; simulation.max_upload_mb)	{name, kind, size} · 413 · 415
DELETE	/simulation/files/{name}	—	{}
POST	/simulation/camera	{file} (null = back to the camera)	{camera_simulation} · 422 not a video or photo
POST	/simulation/face	{person_id}	{person, window_s, events, said} · 404
POST	/simulation/voice	{text} or {file}	{wake_word, stt, transcript, intent, outcome, events, said}

11. DATA AND STORAGE

11.1 SQLite schema (db.py)

Table	Columns	Notes
persons	id, first_name, last_name, can_open_garage, created_at, watchlist , access_days , access_start , access_end , valid_until	
face_embeddings	id, person_id → persons (CASCADE), embedding (BLOB 512 × float32), image_path, source, created_at	source: upload, cli, labeled, auto
voice_profiles	id, person_id (CASCADE), embedding (192 × float32), audio_path, created_at	
unknown_faces	id, embedding, image_path, det_score, created_at, cluster_id	index on cluster_id
sightings	id, ts, track_id, status, person_id (SET NULL), score, quality, image_path, embedding, unknown_id, cluster_id	indexes on ts, person_id and cluster_id
events	id, ts, type, actor, person_id, details (JSON)	index on ts
greetings	person_id (CASCADE), day	primary key (person_id, day)
users	id, username (UNIQUE), password_hash, created_at	
sessions	token_hash (SHA-256), user_id (CASCADE), expires_at	

Migrations: at startup, `Database.init()` runs the schema (`CREATE ... IF NOT EXISTS`). It then adds the columns missing from previous versions (MIGRATIONS list, via `ALTER TABLE ADD COLUMN`), then creates the indexes. The operation is idempotent and tested against a database with the first schema.

11.2 Directory layout

<code>/etc/jarvis/config.yaml</code>	configuration (640 root:jarvis)
<code>/etc/jarvis/jarvis.env</code>	secrets: CAMERA_USER, CAMERA_PASSWORD, PICOVOICE_ACCESS_KEY
<code>/etc/jarvis/tls/</code>	TLS certificate and key
<code>/opt/jarvis/venv</code>	Python 3.11 and dependencies (read-only at runtime)
<code>/var/lib/jarvis/jarvis.db</code>	SQLite database (plus <code>-wal</code> , <code>-shm</code>)
<code>/var/lib/jarvis/faces/<id>/</code>	enrollment photos, including <code>auto_*.jpg</code> (adaptive learning)
<code>/var/lib/jarvis/unknown/<date>/</code>	unknown faces
<code>/var/lib/jarvis/sightings/<date>/</code>	sighting captures
<code>/var/lib/jarvis/voice/<id>/</code>	voice enrollment recordings
<code>/var/lib/jarvis/tmp/</code>	search photos (deleted immediately)
<code>/var/lib/jarvis/models/</code>	YOLO OpenVINO, InsightFace, Vosk, Piper, openWakeWord, ECAPA
<code>/var/lib/jarvis/tts-cache/</code>	synthesized sentences
<code>/run/jarvis/</code>	<code>core.sock</code> , <code>frame.jpg</code> , <code>capabilities.json</code> (tmpfs, 0770 jarvis)

11.3 Retention (maintenance thread, every 6 h)

Data	Duration	Parameter
Unknown faces (images included)	30 days	<code>storage.unknown_retention_days</code>
Sightings (photos included)	90 days	<code>storage.sighting_retention_days</code>
Event log, greetings	180 days	<code>storage.event_retention_days</code>
Expired sessions	immediate	—

12. EVENT LOG AND NOTIFICATIONS

12.1 Event types (events table, Log tab)

Domain	Types
System	core_started, core_stopped, user_created, default_admin_created
Vision	face_recognized, face_unknown, watchlist_seen, access_denied_schedule, face_auto_enrolled
Voice	voice_not_understood, voice_cancel, voice_denied (reasons no_authorized_face, speaker_mismatch)
Garage	garage_pulse (actor, person, intent, source, door state before), garage_pulse_rejected, garage_pulse_simulated (decided during a simulation, relay not driven)
Web: simulation	simulation_file_added, simulation_file_deleted, simulation_camera, simulation_face, simulation_voice
Web: session	login, login_failed (with IP), logout, password_changed, password_change_failed, admin_factory_reset
Web: people	person_created, person_updated, person_deleted, face_added, face_deleted, voice_added, voice_deleted
Web: unknowns	unknown_labeled, unknown_deleted, cluster_labeled, cluster_deleted
Web: traceability	sightings_exported, face_search
Web: actions	ptz_home, say

Everything is also written to **journald** (`journalctl -u jarvis-core -f`).

12.2 Webhook notifications (notify.py)

If `notifications.webhook_url` is set, every event listed in `notifications.events` is sent as a JSON POST:

```
{
  "source": "jarvis",
  "type": "face_unknown",
  "ts": 1790000000.1,
  "datetime": "2026-09-30T09:27:55",
  "unknown_id": 12,
  "cluster_id": 4,
  "sighting_id": 57
}
```

- Emitted types: `face_unknown`, `watchlist_seen`, `access_denied_schedule`, `voice_denied`, `garage_pulse`.
- Sent from a dedicated thread, with a 100-message queue and a `timeout_s` timeout. A network error is only logged: **the core is never blocked**.
- **Home Assistant**: an automation with a *Webhook* trigger (`/api/webhook/<id>`), then a mobile notification or turning on a light.
- **ntfy**: `webhook_url`: `https://ntfy.lan/jarvis`.
- **Node-RED / n8n**: incoming HTTP node.
- F12 network flow: see `docs/diagrams/jarvis-network-flows.pdf`.

13. CONFIGURATION REFERENCE

File: `/etc/jarvis/config.yaml`, validated by pydantic. An unknown key is ignored; an invalid type makes startup fail. Strings accept `${VAR}` and `${VAR:-default}`, read from the environment (`jarvis.env`).

Commented example: `config/config.example.yaml`.

Key	Default	Effect
<code>log_level</code>	<code>INFO</code>	logging level
camera		
<code>rtsp_url</code>	<code>rtsp://user:pass@192.168.50.64:554/stream2</code>	analyzed stream (720p substream recommended)
<code>hw_accel</code>	<code>false</code>	VAAPI decoding
<code>reconnect_delay_s</code>	<code>2.0</code>	first reconnection delay
ptz		
<code>enabled</code>	<code>true</code>	<code>false</code> = fixed camera
<code>host, port, username, password, profile_index</code>	<code>192.168.50.64, 80, admin, empty, 0</code>	ONVIF access
<code>home_preset, return_home_after_s</code>	<code>"1", 30</code>	return to the home position
<code>sound_preset</code>	<code>null</code>	preset targeted by the sound trigger
<code>pan_gain, tilt_gain, zoom_gain</code>	<code>0.6, 0.6, 0.5</code>	controller gains
<code>deadzone, max_speed</code>	<code>0.12, 0.5</code>	dead zone, maximum speed
<code>invert_tilt, zoom_enabled, target_height_ratio</code>	<code>false, true, 0.55</code>	tilt direction, zoom, target size of the person
<code>command_interval_s</code>	<code>0.25</code>	maximum command rate
detector		
<code>model_path, imgsz, conf, tracker</code>	<code>.../yololln_openvino_model, 480, 0.45, bytetrack.yaml</code>	detection and tracking (416 on a 2-core CPU)
faces		
<code>model_name, model_root, providers, det_size</code>	<code>buffalo_s, .../insightface, [CPUExecutionProvider], 320</code>	face models
<code>match_threshold</code>	<code>0.45</code>	recognition threshold
<code>min_face_px, min_det_score, min_sharpness</code>	<code>40, 0.6, 30</code>	quality filters
<code>votes_required, unknown_after_observations</code>	<code>3, 6</code>	per-track voting
<code>recheck_interval_s, max_faces_per_frame</code>	<code>0.4, 2</code>	CPU budget
<code>unknown_dedupe_similarity, unknown_dedupe_window_s</code>	<code>0.5, 3600</code>	unknown deduplication
<code>unknown_cluster_similarity</code>	<code>0.5</code>	visitor clustering
<code>adaptive_enabled, adaptive_min_score, adaptive_min_quality, adaptive_max_per_person, adaptive_novelty_max_similarity</code>	<code>true, 0.6, 0.35, 5, 0.85</code>	adaptive learning

Key	Default	Effect
search_threshold	0.45	search by face
vision		
process_fps, preview_fps, preview_width, frame_path	6, 5, 960, /run/jarvis/frame.jpg	rates, preview
decision		
led_on_s, greeting, auth_window_s, voice_only_after_recognition, listen_after_recognition_s	5, Bonjour {first_name}, 30, true, 0	recognition window: green LED + voice; other LED signals
unknown_message, watchlist_message	null, null	optional sentences
audio		
enabled, input_device, output_device, sample_rate, sound_trigger_rms	true, null, null, 16000, 0	devices (python -m sounddevice lists them)
wakeword		
engine, oww_model, oww_dir, threshold	openwakeword, hey_jarvis_v0.1, .../openwakeword, 0.5	wake word
porcupine_access_key, porcupine_keyword, porcupine_sensitivity	empty, jarvis, 0.6	Porcupine option
stt		
vosk_model_path, listen_timeout_s, max_utterance_s	.../vosk-model-small-fr-0.22, 6, 6	speech recognition
speaker		
enabled, threshold, model_dir	false, 0.35, .../spkrec-ecapa-voxceleb	speaker verification
tts		
piper_model, cache_dir	.../fr_FR-siwis-medium.onnx, .../tts-cache	speech synthesis
commands		
open_phrases, close_phrases, cancel_phrases	3, 3 and 2 phrases	Vosk grammar and intents
hardware		
garage, led_green, led_red: backend, device, channel, active_low	mock, null, 1/2/3, false	outputs
door_sensor: backend, device, line, active_means_closed	none, null, 0, true	door sensor
pulse_ms, cooldown_s	500, 5	garage pulse
storage		
data_dir	/var/lib/jarvis	data root
unknown_retention_days, sighting_retention_days, event_retention_days	30, 90, 180	GDPR retention
simulation		
loop, drive_relay, require_window, max_upload_mb	true, false (DANGER), true, 100	camera and voice simulation (§18)
api		
host, port, session_hours, cookie_secure, max_upload_mb	127.0.0.1, 8000, 12, true, 15	web server
notifications		

Key	Default	Effect
webhook_url, events, timeout_s	null, 5 types, 5	webhooks
control		
socket_path	/run/jarvis/core.sock	API → core socket

14. OPERATIONS, PERFORMANCE, TESTS

14.1 Operations

```
systemctl status jarvis-core jarvis-api      # service status
journalctl -u jarvis-core -f                # live log
sudo systemctl restart jarvis-core          # after editing config.yaml
sudo -u jarvis sqlite3 /var/lib/jarvis/jarvis.db ".backup /root/jarvis-$(date +%F).db" # online backup
./scripts/jarvis-regen-sbom.sh              # up-to-date software inventory (SBOM)
```

To back up: /etc/jarvis/ and /var/lib/jarvis/, excluding models/ and tts-cache/, which are regenerated.

14.2 Measured and estimated performance

Stage	Cost
YOLO11n OpenVINO + ByteTrack, imsz 480	60 to 100 ms per frame on a Haswell i5 (89 ms measured on a loaded development PC)
SCRFD-500M + ArcFace per face region	20 to 60 ms (59 ms measured)
H.264 720p decoding at 15 frames/s	about 10 % of one core
openWakeWord · Vosk (while listening)	about 5 % · about 20 % of one core
Piper medium	about 0.3 s of compute per second of speech, then cached
Full pipeline (test video, 6 people)	10 to 15 frames/s analyzed on the development workstation

Default target: 6 analyses per second. On an i5-4570T (2 cores): vision.process_fps: 4 and detector.imsz: 416.

14.3 Automated tests

201 tests (pytest), none of which needs a model or hardware. Main files:

File	Tests	Scope
tests/test_logic.py	35	configuration, command grammar and parsing, relay protocols, PTZ controller, target selection, gallery, voting, decision rules
tests/test_features.py	28	access rules, migrations, sightings, clustering, watchlist, schedules, notifications, adaptive learning, CSV, statistics, face search, retroactive labeling, threading.Thread attribute guard
tests/test_admin.py	13	settings catalog and hot reload, audit with diffs and hash chain, sessions, idle timeout
tests/test_api.py	9	authentication, login rate limiting, CSRF, person and face lifecycle, control socket
tests/test_secrets.py	7	precedence parameter > environment > file, write-only secrets, token migration, webhook bearer
tests/test_cli.py	6	cmd2 shell commands and exit codes
tests/test_scripts.py	6	jarvis-hwinfo JSON/verdict, jarvis-motd, SSH banner, jarvis-deploy.sh, jarvis-common.sh error handling
tests/test_recording.py	5	pre-roll/post-roll clips, FIFO quota, time-lapse, Range requests
tests/test_sysinfo.py	5	hardware detection, auto-tuning, capabilities snapshot
tests/test_version_cert.py	5	single version source, /api/me version, bump rules, jarvis-cert local lifecycle and input validation
tests/test_default_admin.py	3	default admin/admin, forced password change, password policy
tests/test_simulation.py	13	domain of every settings group, wake word token, voice chain with fallback, simulated pulses never drive the relay (unless drive_relay), camera simulation marks events, upload validation (size, type, path traversal)

File	Tests	Scope
tests/test_mqtt.py	2	Home Assistant discovery, MQTT bridge against a real broker

The Ansible deployment has its own end-to-end test (`deploy/ansible/tests/jarvis-container-test.sh`): an Ubuntu 26.04 systemd container reached over SSH, full deployment with postflight checks, then an idempotence re-run.

Lightweight environment: `pip install -r requirements-dev.txt`, then `PYTHONPATH=. pytest`. No model or hardware required.

The complete chain has also been validated **with the real models**, on a test video containing several faces:

1. detection, tracking, unknown faces, clustering, sightings with photo;
2. labeling of a cluster through the API and retroactive identification;
3. recognition on the next pass;
4. CSV export and search by face;
5. walkthrough of the interface in Chrome (Playwright), without any JavaScript error.

15. KNOWN LIMITATIONS

- **No liveness detection** (anti-spoofing): a good-quality photo can fool the face part. Countermeasures: voting over several frames, voice verification (`speaker.enabled`), the short recognition window (30 s), IR at night (most screens do not show up under IR). Possible evolution: a dedicated anti-spoofing model. A lightweight heuristic was deliberately ruled out: it would give a false sense of security.
- **Speaker verification does not detect a replayed** recording.
- **Adaptive learning**: a confusion with a score ≥ 0.6 would be learned (cap of 5, additions visible and removable).
- **Incremental clustering**: arrival order influences the clusters. Two clusters of the same person can coexist. Simply label both of them as the same person.
- **Door without sensor**: "open" on an open door closes it again (pulse input of the Novomatic).
- **Convenience system**, not certified access control: keep a second lock for access to the house.
- **GDPR**: strictly domestic use, camera limited to the property, visitors informed, consent of household members (see `docs/ARCHITECTURE.md` §6).

16. DEPLOYMENT AND OPERATIONS TOOLING

16.1 Central version file (jarvis/VERSION)

- jarvis/VERSION is the **single source of truth** for the version (Semantic Versioning MAJOR.MINOR.PATCH[-pre]). jarvis/__init__.py reads it into __version__, and pyproject.toml declares dynamic = ["version"] with [tool.setuptools.dynamic] version = {file = "jarvis/VERSION"}.
- Consumers: jarvis version in the shell, GET /api/me ("version", authenticated users only), the FastAPI app, jarvis-cert --version (via jarvis_version in scripts/lib/jarvis-common.sh), and every file header (Project : jarvis-home (version: jarvis/VERSION)).
- **Never edit it by hand:** scripts/jarvis-bump-version.py patch|minor|major|X.Y.Z (or make bump PART=patch|minor|major|X.Y.Z) validates SemVer, refuses to go backwards and moves the [Unreleased] section of CHANGELOG.md under a dated [x.y.z] heading. Options --show and --dry-run. Exit codes: 0 success, 2 usage error or invalid version, 4 VERSION or CHANGELOG missing, 5 version not greater than the current one.

16.2 TLS manager (scripts/jarvis-cert.sh, installed as /usr/local/sbin/jarvis-cert)

It manages the certificate served by nginx at the stable paths /etc/jarvis/tls/jarvis.crt (certificate + chain) and /etc/jarvis/tls/jarvis.key, in one of two modes remembered in /etc/jarvis/tls/jarvis-cert.conf:

Mode	Use	Mechanism
local (default of the installers)	LAN only, no public domain	Private "Jarvis Local CA" (ECDSA P-384, 10 years, name-constrained to local names and private IPv4 ranges) signing a server certificate (ECDSA P-256, 397 days) for .local/.lan/.home.arpa names and LAN IPs. Import the CA once on client devices (export-ca). Revocation uses a real CA database and CRL.
letsencrypt	public domain name	certbot obtains an ECDSA certificate. The default DNS-01 challenge needs no inbound port open on the Internet (Jarvis stays LAN-only); HTTP-01 (--challenge http) requires port 80 reachable from the Internet (nginx serves /.well-known/acme-challenge/ from /var/www/letsencrypt).

Subcommand	Role
issue --mode local [--name NAME]... [--ip IPV4]... [--days N] [--new-ca]	issue a certificate signed by the local CA
issue --mode letsencrypt --domain FQDN --email ADDR [--challenge dns http] [--dns-plugin NAME --dns-credentials FILE] [--staging] [--install-deps]	obtain a Let's Encrypt certificate
renew [--force] [--days-before N]	renew when close to expiry (default 30 days)
status [--cert FILE] [--warn-days N] [--remote HOST[:PORT]] [--json] [--no-revocation]	dates, remaining time, revocation, chain (read-only, no root needed)
revoke [--reason unspecified keyCompromise superseded cessationOfOperation] [--yes]	revoke the current certificate
export-ca [--out FILE]	export the local CA certificate for the clients

Global options: --tls-dir DIR (env JARVIS_TLS_DIR), --no-reload (env JARVIS_CERT_NO_RELOAD=1), --yes (env ASSUME_YES=1), --no-color. Root is required for any change in /etc/jarvis/tls.

Exit codes: 0 success / certificate valid, 1 runtime failure, 2 usage error, 3 root required, 4 missing prerequisite, 5 invalid configuration or option, 6 network failure (ACME, CRL/OCSP), 7 certificate unusable (expired, not yet valid, revoked, key mismatch, bad chain), 8 valid but expiring within --warn-days (default 30), 130 interrupted.

Automatic renewal: deploy/systemd/jarvis-cert-renew.timer fires twice a day (04:17 and 16:17, RandomizedDelaySec=1h, Persistent=true) and starts the oneshot jarvis-cert-renew.service, which runs

jarvis-cert renew --days-before 30 then jarvis-cert status --warn-days 7: an invalid result fails the unit (visible in systemctl --failed and in the MOTD).

16.3 Web front end: nginx and Anubis

- `deploy/nginx/jarvis.conf` is identical on every host. The two host-specific values live in generated snippets (templates in `deploy/nginx/snippets/`): `/etc/nginx/snippets/jarvis-upstream.conf` (upstream `jarvis_front`: the API `127.0.0.1:8000` by default, Anubis `127.0.0.1:8923` when deployed by Ansible) and `/etc/nginx/snippets/jarvis-server-name.conf` (`server_name`, must match the certificate names).
- Path: client → nginx :443 → `jarvis_front` → API. `/api/stream.mjpg` (no buffering, 1 h read timeout) and `/metrics` go straight to the API; `/api/login` is rate-limited (`limit_req`, 429) and then goes through `jarvis_front`.
- **Anubis** (`anubis@jarvis.service`, files in `deploy/anubis/`): anti-bot proxy with a proof-of-work challenge (`DIFFICULTY=4`), listening on `127.0.0.1:8923` and forwarding to `http://127.0.0.1:8000`. Its policy `jarvis.botPolicies.yaml` denies pathological clients, AI crawlers and agents, headless browsers, search engines and link-preview bots (HTTP 403), allows `/metrics`, challenges browser-like clients once (cookie), and passes other non-browser clients through (the API still requires a session or a token). The `Ed25519` signing key `/etc/anubis/jarvis.key` is handed over as a systemd credential by the drop-in `anubis@jarvis.service.d/10-jarvis.conf`.
- **No indexing**: nginx serves `/robots.txt` itself (`User-agent: * / Disallow: /`), adds `X-Robots-Tag: noindex, nofollow, noarchive, nosnippet, noimageindex, notranslate` to every response, and a map on `$http_user_agent` (`$jarvis_denied_bot`) returns **403** to AI crawlers/agents (GPTBot, ClaudeBot, CCBot, PerplexityBot, Bytespider...) and search-engine robots (Googlebot, Bingbot...) before they reach Anubis.

16.4 Capability snapshot and SSH MOTD

- At startup, the core detects the hardware (`detect_capabilities`), applies the automatic performance choices (`apply_auto_performance`), then `write_capabilities_snapshot` (`jarvis/core/sysinfo.py`) publishes `/run/jarvis/capabilities.json` next to the control socket: `{"ts", "capabilities", "performance"}` (CPU model and flags, GPUs, OpenVINO/ONNX devices, VAAPI codecs). The file is written atomically, world-readable (mode 0644, no secret), and a failure is only logged.
- `deploy/motd/jarvis-motd` (installed as `/usr/local/bin/jarvis-motd`, called at every SSH login by the `update-motd` hook `/etc/update-motd.d/10-jarvis`, capped at 5 s) prints the banner, live metrics, the hardware support read from `capabilities.json` (or probed directly), the state and resource usage of the Jarvis units, the TLS certificate and the security status. It always exits 0 (2 on a usage error): a MOTD must never block a login.

16.5 Hardware inventory (`scripts/jarvis-hwinfo.sh`)

`jarvis-hwinfo [--section NAME]... [--json] [--bench] [--no-color]` inventories the machine before or after installation (no Jarvis component needed): system, CPU (ISA extensions such as AVX2, AVX-512, VNNI), NPU, GPU (VAAPI, OpenCL, Vulkan, OpenVINO/ONNX devices), memory, storage (SMART, optional 256 MiB sequential benchmark), network, USB (recognized Jarvis modules: ReSpeaker, CH340 relay, FT232R...), audio and sensors. It ends with a suitability verdict against `REQUIREMENTS.md`. Exit codes: 0 requirements met, 2 usage error, 8 machine below a Jarvis requirement.

16.6 Shared bash library (`scripts/lib/jarvis-common.sh`)

Sourced (never executed) by every Jarvis shell script, installed as `/usr/local/lib/jarvis/jarvis-common.sh`. It provides strict mode (`set -euo pipefail`) with an ERR trap reporting the failing command, file and line; colored logging (`log_info`, `log_ok`, `log_warn`, `log_error`, `log_step`, `log_debug`), disabled when stderr is not a terminal, with `NO_COLOR` or `JARVIS_COLOR=never`; `die MESSAGE [CODE]`; `require_root`, `require_cmd`, `require_file`; LIFO cleanup registration (`on_exit`); `confirm`, `jarvis_version`, `is_true`. Standard exit codes shared by all scripts: 0 `E_OK`, 1 `E_RUNTIME`, 2 `E_USAGE`, 3 `E_PRIV`, 4 `E_DEPS`, 5 `E_CONFIG`, 6 `E_NETWORK`, 7 `E_STATE`, 8 `E_WARN`, 130 interrupted. `JARVIS_DEBUG=1` enables `log_debug` and `xtrace`.

16.7 Ansible deployment (deploy/ansible/)

The reference deployment of a host is the Ansible playbook in `deploy/ansible/` (`site.yml`, `jarvis-deploy.sh`), which installs Jarvis, nginx, Anubis, TLS (`jarvis_tls_mode`: `local` by default, `letsencrypt` with the DNS-01 challenge by default), the firewall, fail2ban, hardening, the MOTD and monitoring. The step-by-step procedure is in `INSTALL.md`, and the playbook reference (variables, TLS modes, roles, troubleshooting) is in `deploy/ansible/README.md`.

17. LICENSE PLATE RECOGNITION AND VEHICLE AUTOMATION

Disabled by default (`plates.enabled`). When it is enabled, Jarvis reads the license plates of the vehicles in view and can open, and optionally close, the garage for registered plates. Every read and every decision is traced in the event log (`plate_*` events, garage pulses with `source: plate`), in the `plate_reads` table (with the vehicle snapshot) and in the service log.

17.1 Pipeline

1. **Detection:** the person detector also tracks the COCO vehicle classes car, motorcycle, bus and truck (`plates.vehicle_classes`), with the same ByteTrack instance. Persons keep driving the PTZ and the face recognition, and vehicles go to the plate reader. The preview draws vehicles in orange, with their plate and direction.
2. **Reading:** `PlateReader` (`jarvis/vision/plates.py`) handles one read per vision cycle at most. - **Which vehicle:** the largest vehicle that is due (`read_interval_s`) and at least `min_vehicle_px` wide. - **How:** its crop goes to fast-alpr, a YOLOv9 plate detector followed by the CCT "global" OCR, which also predicts the country or region. It runs on ONNX Runtime on the CPU, in about 30 ms per read on a recent CPU, and several times that on the Haswell.
3. **Confirmation:** the plate is confirmed after `votes_required` identical normalized reads (default 2), each with a mean character confidence of at least `min_confidence` (default 0.80). Only the confirming read keeps its snapshot, stored under `/var/lib/jarvis/plates/YYYYMMDD/`.
4. **Direction:** it comes from the vehicle box. - Area growth of at least `approach_growth` ($\times 1.15$) over 2 s means **approaching**. - Shrinkage to `leave_shrink` ($\times 0.85$) or less means **leaving**. - A vehicle unseen for 3 s is **gone**.

Normalization (`jarvis/core/plates.py`): the plate goes through Unicode NFKC and upper case, and only letters and digits of any script are kept. "AB-123-CD", "ab 123 cd" and "AB·123·CD" are the same plate. Comparison is exact: there is no O/0 or I/1 tolerance, which would widen the set of plates that open the garage.

17.2 Decisions (`DecisionEngine._on_vehicle`)

Situation	Conditions	Action and trace
Plate confirmed	—	<code>plate_reads</code> row + <code>plate_read</code> event (status known / unknown / disabled / expired); notification <code>plate_recognized</code> , or <code>plate_unknown</code> / <code>plate_refused</code> (<code>unknown_notify</code>)
Known vehicle approaching	<code>auto_open</code> ; plate enabled and not expired; the linked person (if any) has <code>can_open_garage</code> and is inside their access rules; door sensor = closed	pulse (<code>garage_pulse</code> , <code>source: plate</code>), read action opened; any pending close is cancelled (<code>garage_close_cancelled</code>)
Door open already	—	no pulse (it would close the single-button motor): <code>plate_open_refused</code> reason <code>already_open</code>
Door state unknown	no door sensor	no pulse: <code>plate_open_refused</code> reason <code>door_state_unknown</code>
Known vehicle leaving , then gone	<code>auto_close</code> (opt-in)	<code>garage_close_scheduled</code> ; the close runs once the scene (no person, no vehicle) stayed clear for <code>close_delay_s</code> , and any presence restarts the countdown
Pending close due	door sensor = open	pulse, read action closed, <code>garage_closed_vehicle_left</code> ; otherwise <code>plate_close_refused</code> (<code>already_closed</code> , <code>door_state_unknown</code> , or <code>scene_never_clear</code> after <code>close_give_up_s</code>)

Every refusal is logged once per vehicle and reason. The other reasons are `plate_unknown`, `plate_disabled`, `plate_expired`, `person_not_allowed`, `schedule_<reason>`, `auto_open_disabled` and `auto_close_disabled`.

DANGER

— automatic closing. An unattended closing door can injure a person or an animal, or damage a vehicle. `plates.auto_close` is therefore off by default. Enable it only when: - the Novomatic obstacle detection (force limitation) is tested and working; - a safety photocell is fitted across the opening; - the camera covers the whole door area, because the "clear scene" check only sees what the camera sees.

17.3 Registry, API and interface

- **Tables:** `plates` holds the plate (normalized, unique), display text, country, label, optional linked person, enabled flag and expiration date. `plate_reads` holds the confirmed reads.
- **API:**
- `GET/POST /api/plates`, `PATCH/DELETE /api/plates/{id}` (audited: `plate_added`, `plate_updated` with the before/after diff, `plate_deleted`);
- `GET /api/plate-reads` (date range, daily time window, plate fragment, status).
- **Vehicles page:** the registry (add, enable or disable, delete) and the read history, with the snapshot, direction and action.
- **Settings:** the License plates group; retention in Data retention (GDPR), `plate_read_retention_days`.
- **Hot changes:** registry changes apply immediately, because the decision engine reads the registry at each read. Enabling or disabling the feature requires a core restart.

17.4 Models and limits

- **Models:** `jarvis setup-models` downloads the two ONNX models once, when `plates.enabled` is true. They go into `/var/lib/jarvis/.cache/open-image-models` and `.../fast-plate-ocr`, so plate reading then runs **offline**.
- **Libraries:** `fast-alpr`, `fast-plate-ocr` and `open-image-models` are MIT-licensed.
- **Model weights:** their licence is not stated on the project pages (see `THIRD-PARTY-NOTICES.md`).
- **Camera placement:** plates need pixels. Aim for a plate at least **100-130 px wide** in the image, at the reading distance.
- **Viewing angle:** at most about 30° horizontal and vertical.
- **At night:** IR, and a short shutter to avoid motion blur.
- **Second camera:** a camera placed for faces (height, angle) is often poorly placed for plates; a dedicated fixed camera may be needed (see `docs/CAMERA-SELECTION.md`).
- **Spoofing:** a plate is not a secret, and anyone can copy one. Plate-based opening is a convenience, not authentication.
- Limit each registered plate with an expiration date and the linked person's access rules.
- Keep `unknown_notify` on.
- Review the plate reads regularly.

18. SETTINGS TABS AND SIMULATION

18.1 Tabbed settings

The Settings page has two navigation levels. Each **domain tab** groups the parameter groups of one domain; each group is a **sub-tab**. The mapping is DOMAINS in `jarvis/config/catalog.py`, and a test checks that every group belongs to exactly one domain.

Domain tab	Sub-tabs (groups)
Camera & vision	Camera stream, PTZ camera, Person detection, Performance
Recognition	Face recognition, Unknown visitors, Adaptive learning, Search by face, License plates
Access & voice	Decision and access, Voice
Recording	Video recording, Time-lapse
Integrations	Smart home (MQTT), Notifications, Monitoring (Grafana)
System	Web interface, Logging, Data retention (GDPR), Secrets
Simulation	Camera simulation, Voice simulation

- **Unsaved edits:** a dot marks each domain tab that holds unsaved edits. The save bar applies them all at once, whatever the tab.
- **Remembered tab:** the last opened sub-tab is kept in the browser (`localStorage`, `jarvis.settings.tab`).
- **Cards:** the camera card (stream test, annotated preview) follows Camera stream, the monitoring card follows Monitoring, and the simulation cards follow their sub-tabs.

18.2 Camera simulation

Plays an uploaded file **instead of the RTSP stream**, through the whole real pipeline: person detection, tracking, face recognition, plates, preview and recordings.

- **Inputs:** a video (`.mp4`, `.mkv`, `.mov`, `.avi`, `.m4v`; `.webm` counts as a recording) is paced at its own frame rate and looped if `simulation.loop` is on; a photo (`.jpg`, `.png`...) is repeated at 5 frames/s.
- **Switching:** `RtspCamera.simulate(path)` switches the source without restarting the core; "Back to the camera stream" (or `{file: null}`) returns to RTSP.
- **Not saved:** the simulation is not persisted. A core restart goes back to the camera.
- **Files:** uploads go to `storage.simulation_dir` (`/var/lib/jarvis/simulation`). The API accepts a bare file name only (no path), checks the extension and caps the size (`simulation.max_upload_mb`, 100 MB; nginx allows 100 MB on this route only).

18.3 Voice simulation

Tests the voice chain **without a microphone** (`jarvis/voice/simulate.py`):

1. **Recognize a person:** `simulate_face` injects a `FaceRecognized` event for the chosen person. As with a real recognition, it opens the recognition window (green LED, greeting).
2. **Say a command**, typed or recorded:
 - **typed:** the text must start with the wake word ("Jarvis, ouvre la porte du garage"). The command part is synthesized by Piper, then transcribed by Vosk with the live restricted grammar. Without the Piper or Vosk model, the text is parsed directly (the result says so).
 - **recording:** an uploaded audio file (any format FFmpeg reads) goes through the configured wake word engine, then Vosk transcribes what follows the wake word.
3. **Result:** each stage is shown (wake word heard, speech recognition, transcript, command, outcome), with the decisions logged by the engine and what Jarvis would have said.

The live rules apply: no wake word → ignored; with `simulation.require_window` (default) and `decision.voice_only_after_recognition`, a command outside a recognition window is ignored ("microphone asleep"). Turn `require_window` off to test the voice chain alone.

18.4 Safety: the relay

A garage pulse decided during a simulation does not drive the relay. Simulated faces and voice commands carry `simulated=True`, and every event produced while the camera plays a file is also treated as simulated. The decision engine logs `garage_pulse_simulated` (with the intent and the person) and lights the green LED, but sends nothing to the relay.

`simulation.drive_relay` (DANGER, off by default) lifts this guard for an end-to-end test with the real door. Turn it back off afterwards.

Every simulation action is audited with its author (`simulation_*` events).

APPENDIX A GLOSSARY

Technical terms and abbreviations used in this document.

ACME

Automatic Certificate Management Environment (RFC 8555): protocol used by certbot to obtain certificates from Let's Encrypt, with an HTTP-01 or DNS-01 challenge.

Adaptive learning

Automatic, capped enrichment of a person's gallery with their best captures, when they are recognized with certainty.

AEC

Acoustic Echo Cancellation: removal of the loudspeaker signal from the microphone signal. Performed here by the XMOS XVF-3000 chip of the ReSpeaker.

Alloy / Promtail

Grafana agents that ship log files and the systemd journal to Loki. Grafana has deprecated Promtail in favor of Alloy.

Anubis

Proof-of-work reverse proxy that keeps crawlers, AI agents and scrapers away from the web UI (instance anubis@jarvis on 127.0.0.1:8923).

API (REST)

HTTP programming interface in which each resource (person, sighting...) has a URL and is handled with the GET, POST, PATCH and DELETE verbs.

ArcFace

Neural network and loss function (additive angular margin) that turn an aligned face into a 512-dimension vector, compared by cosine similarity.

Argon2id

Password hashing function resistant to GPU attacks (winner of the Password Hashing Competition, RFC 9106).

Authorization window

Delay (60 s) during which an authorized person recognized by the camera can command the garage by voice.

AVX2

256-bit vector instruction set of Intel processors since Haswell, used by OpenVINO and ONNX Runtime. Missing from the Celeron and Pentium parts of that generation.

BOM

Bill of Materials: exhaustive list of the hardware components, with quantities and references.

ByteTrack

Multi-object tracking algorithm that associates detections from one frame to the next by IoU, in two passes (high then low confidence), with Kalman filter prediction.

CA (certificate authority)

Entity that signs certificates. Jarvis either runs a private, name-constrained "Jarvis Local CA" or uses Let's Encrypt.

certbot

ACME client of the EFF used by jarvis-cert to obtain and renew Let's Encrypt certificates.

CH340

WCH USB-to-serial converter chip fitted to the LCUS relay boards.

Cooldown

Minimum delay enforced between two pulses of the garage relay (5 s by default).

CSP

Content Security Policy: HTTP header that restricts the sources of scripts, images and frames of a page.

CSRF

Cross-Site Request Forgery: attack that makes a logged-in user perform an action from a third-party site. Countered here by the mandatory X-Jarvis header and the SameSite=Strict cookie.

CTS / DTR

Control lines of a serial link: CTS (Clear To Send) is an input, DTR (Data Terminal Ready) an output. CTS is used as the logic input of the door sensor.

Dead zone

Tolerance around the image center inside which the PTZ does not move (12 %).

Deduplication

Removal of redundant records: an unknown face very close to an unknown face seen within the hour is not recorded again.

DNS-01

ACME challenge proven by publishing a TXT record in the public DNS zone through the DNS provider API. It needs no inbound port open on the Internet.

ECAPA-TDNN

Speaker verification neural network (SpeechBrain) that produces a 192-dimension voiceprint.

Embedding

Representation of a face or a voice as a numeric vector; two close embeddings probably belong to the same person.

fail2ban

Log-driven intrusion prevention: bans the source addresses that repeatedly fail SSH or web logins.

FFmpeg

Audio/video decoding and encoding library and tool, used for the RTSP stream and for converting the voice recordings.

FT232R / FT232RL

FTDI USB-to-serial converter chip with TTL levels; its CTS# pin reads the door sensor. Its internal pull-up (about 200 kΩ) is weak.

Gallery

Set of reference embeddings of the enrolled persons, loaded in memory.

GDPR

General Data Protection Regulation (EU 2016/679). Faces and voices are biometric data (Art. 9).

Grafana

Dashboard server of the optional monitoring stack (Loki and Prometheus data sources).

H.264 / H.265

Video compression standards (AVC / HEVC). The analyzed stream must be H.264, the only one the Intel HD 4600 decodes in hardware.

Half-duplex

Operating mode in which the microphone is ignored while the loudspeaker is talking.

Home Assistant

Open-source home automation controller; consumes the Jarvis MQTT discovery messages and webhooks.

HSTS

HTTP Strict Transport Security: forces the browser to use HTTPS for the site.

Identification by vote

Several matching observations (3) are required before a person is declared recognized.

InsightFace

Open-source face analysis library (SCRFD detection, ArcFace recognition).

IoU

Intersection over Union: ratio between the common area and the union area of two boxes; measures their overlap.

IR

Infrared: invisible lighting that gives a black-and-white image at night.

journald

systemd logging service (journalctl command).

Kaldi

Speech recognition toolkit on which Vosk is built.

Laplacian (variance of the)

Image sharpness measure: a low variance means a blurred image.

LCUS-4

4-relay board driven over USB through a CH340, serial frame A0 channel state checksum.

Let's Encrypt

Free public certificate authority, reached with the ACME protocol.

MJPEG

Video stream made of a sequence of JPEG images, played directly by a browser.

MQTT

Lightweight publish/subscribe messaging protocol (TCP 1883, 8883 with TLS) used by the optional smart-home bridge.

node_exporter

Prometheus agent that exposes the host metrics (CPU, memory, disk, network) on port 9100.

ONNX / ONNX Runtime

Open AI model format and optimized inference engine (Microsoft).

ONVIF

IP camera interoperability standard (discovery, streaming, PTZ). Profile S: video and PTZ.

OpenVINO

Intel inference engine optimized for its processors.

openWakeWord

Open-source wake-word detector (ONNX models).

Piper

Offline neural text-to-speech engine (VITS exported to ONNX).

Porcupine

Commercial wake-word detector from Picovoice (free key for personal use).

Preset

PTZ position stored in the camera (pan, tilt, zoom).

Prometheus

Metrics collection server; scrapes the Jarvis /metrics endpoint and node_exporter.

Proportional controller

Control loop whose command is proportional to the measured error (here, the offset between the person and the image center).

PTZ

Pan-Tilt-Zoom: camera motorized in pan, tilt and zoom.

Pull-up

Resistor that pulls a logic input to the high state when nothing drives it low.

Pulse

Short closure (500 ms) of the relay contact, equivalent to pressing the push button.

Reed switch

Switch with flexible blades operated by a magnet; used as a door opening sensor.

Restricted grammar

Closed list of phrases imposed on the Vosk decoder: everything else is rejected as [unk], which makes recognition very robust to noise.

Retroactive identification

Assignment, after the fact, of an unknown visitor's sightings to a person when their cluster is labeled.

RTSP

Real Time Streaming Protocol (RFC 2326/7826): streaming protocol of the camera video.

SBOM

Software Bill of Materials: exhaustive inventory of the software components, versions and licenses.

SCRFD

Fast InsightFace face detector (Sample and Computation Redistribution).

Sighting

Time-stamped record of an identification: date, time, identity or unknown, score, photo.

Similarity (cosine)

Cosine of the angle between two vectors, from -1 to 1; equal to the dot product of normalized vectors.

Similarity threshold

Minimum cosine between two embeddings to conclude that they belong to the same person (0.45 for faces).

SQLite (WAL)

Database embedded in a single file; Write-Ahead Log mode allows concurrent reads and a write.

syslog

Standard log transport (RFC 3164, UDP or TCP 514) to an rsyslog or syslog-ng server.

systemd

Linux service manager (jarvis-core, jarvis-api units, timers).

TLS / HTTPS

Encryption of web traffic (Transport Layer Security).

tmpfs

File system held in RAM (/run).

Track

Sequence of detections of the same person over time, identified by a number (ByteTrack).

TTS

Text-To-Speech, speech synthesis.

udev

Linux device manager; its rules give stable names (/dev/jarvis-relay).

Unix socket

Local inter-process communication channel represented by a file (/run/jarvis/core.sock).

Unknown cluster

Automatic grouping of the visits of the same unknown visitor.

VAAPI

Video Acceleration API: hardware video decoding on Intel GPUs.

VITS

End-to-end neural speech synthesis architecture used by Piper.

Vosk

Offline speech recognition engine built on Kaldi.

Wake word

Word or phrase ("hey jarvis") that activates command listening.

Watchlist

Flagged persons whose recognition triggers an alert and never grants access.

Webhook

HTTP request sent automatically to another service when an event occurs.

YOLO

You Only Look Once: family of single-pass object detectors; YOLO11n is the nano variant.

APPENDIX B BIBLIOGRAPHY

Further reading. Titles are given as published; ISBNs are those of the edition cited and are omitted when they could not be verified.

B.1 Books in English

1. Anderson, Ross. *Security Engineering: A Guide to Building Dependable Distributed Systems*. 3rd ed. Wiley, 2020. ISBN 978-1-119-64278-7.
2. Åström, Karl Johan; Murray, Richard M. *Feedback Systems: An Introduction for Scientists and Engineers*. Princeton University Press, 2008. ISBN 978-0-691-13576-2.
3. Bishop, Christopher M. *Pattern Recognition and Machine Learning*. Springer, 2006. ISBN 978-0-387-31073-2.
4. DeJonghe, Derek. *NGINX Cookbook: Advanced Recipes for High-Performance Load Balancing*. 3rd ed. O'Reilly, 2024. ISBN 978-1-098-15843-9.
5. Geerling, Jeff. *Ansible for DevOps: Server and Configuration Management for Humans*. 2nd ed. Leanpub, 2020.
6. Gonzalez, Rafael C.; Woods, Richard E. *Digital Image Processing*. 4th ed. Pearson, 2018. ISBN 978-0-13-335672-4.
7. Goodfellow, Ian; Bengio, Yoshua; Courville, Aaron. *Deep Learning*. MIT Press, 2016. ISBN 978-0-262-03561-3.
8. Huang, Xuedong; Acero, Alex; Hon, Hsiao-Wuen. *Spoken Language Processing: A Guide to Theory, Algorithm, and System Development*. Prentice Hall, 2001. ISBN 978-0-13-022616-7.
9. Huyen, Chip. *Designing Machine Learning Systems: An Iterative Process for Production-Ready Applications*. O'Reilly, 2022. ISBN 978-1-098-10796-3.
10. Jurafsky, Daniel; Martin, James H. *Speech and Language Processing*. 2nd ed. Pearson Prentice Hall, 2009. ISBN 978-0-13-187321-6.
11. Kaehler, Adrian; Bradski, Gary. *Learning OpenCV 3: Computer Vision in C++ with the OpenCV Library*. O'Reilly, 2016. ISBN 978-1-4919-3799-0.
12. Kerrisk, Michael. *The Linux Programming Interface: A Linux and UNIX System Programming Handbook*. No Starch Press, 2010. ISBN 978-1-59327-220-3.
13. Li, Stan Z.; Jain, Anil K. (eds.). *Handbook of Face Recognition*. 2nd ed. Springer, 2011. ISBN 978-0-85729-931-4.
14. Pivotto, Julien; Brazil, Brian. *Prometheus: Up & Running: Infrastructure and Application Performance Monitoring*. 2nd ed. O'Reilly, 2023. ISBN 978-1-098-13114-2.
15. Pulver, Tim. *Hands-On Internet of Things with MQTT*. Packt, 2019. ISBN 978-1-78934-178-2.
16. Rabiner, Lawrence; Juang, Biing-Hwang. *Fundamentals of Speech Recognition*. Prentice Hall, 1993. ISBN 978-0-13-015157-5.
17. Ramalho, Luciano. *Fluent Python: Clear, Concise, and Effective Programming*. 2nd ed. O'Reilly, 2022. ISBN 978-1-4920-5635-5.
18. Ristić, Ivan. *Bulletproof TLS and PKI: Understanding and Deploying SSL/TLS and PKI to Secure Servers and Web Applications*. 2nd ed. Feisty Duck, 2022. ISBN 978-1-907117-09-1.
19. Stevens, W. Richard; Fenner, Bill; Rudoff, Andrew M. *UNIX Network Programming, Volume 1: The Sockets Networking API*. 3rd ed. Addison-Wesley, 2003. ISBN 978-0-13-141155-5.
20. Szeliski, Richard. *Computer Vision: Algorithms and Applications*. 2nd ed. Springer, 2022. ISBN 978-3-030-34371-2.
21. Taylor, Paul. *Text-to-Speech Synthesis*. Cambridge University Press, 2009. ISBN 978-0-521-89927-7.
22. Ward, Brian. *How Linux Works: What Every Superuser Should Know*. 3rd ed. No Starch Press, 2021. ISBN 978-1-7185-0040-2.

B.2 Books in French

1. Azencott, Chloé-Agathe. *Introduction au Machine Learning*. 2nd ed. Dunod, 2022. ISBN 978-2-10-083476-1.
2. Blaess, Christophe. *Développement système sous Linux : ordonnancement multitâche, gestion mémoire, communications, programmation réseau*. 5th ed. Eyrolles, 2019. ISBN 978-2-212-67760-7.
3. Géron, Aurélien. *Deep Learning avec Keras et TensorFlow : mise en œuvre et cas concrets*. 3rd ed. Dunod, 2024. ISBN 978-2-10-084769-3.
4. Ghernaouti, Solange. *Cybersécurité : analyser les risques, mettre en œuvre les solutions*. 8th ed. Dunod, 2025. ISBN 978-2-10-087183-4.
5. Haton, Jean-Paul; Cerisara, Christophe; Fohr, Dominique; Laprie, Yves; Smaïli, Kamel. *Reconnaissance automatique de la parole : du signal à son interprétation*. Dunod, 2006. ISBN 978-2-10-005842-6.
6. Le Cun, Yann. *Quand la machine apprend : la révolution des neurones artificiels et de l'apprentissage profond*. Odile Jacob, 2019. ISBN 978-2-7381-4931-2.
7. Pujolle, Guy. *Les Réseaux*. 2024-2026 edition (10th ed.). Eyrolles, 2024. ISBN 978-2-416-01433-8.

8. Rohaut, Sébastien; Banquet, Philippe. *Linux : maîtrisez l'administration du système*. 7th ed. Éditions ENI, 2024. ISBN 978-2-409-04366-6.
9. Swinnen, Gérard. *Apprendre à programmer avec Python 3*. 3rd ed. Eyrolles, 2012. ISBN 978-2-212-13434-6.
10. Tanenbaum, Andrew S.; Wetherall, David J. *Réseaux*. 5th ed. Pearson, 2011. ISBN 978-2-7440-7521-6.

B.3 Standards, specifications and papers

1. Deng, J.; Guo, J.; Xue, N.; Zafeiriou, S. "ArcFace: Additive Angular Margin Loss for Deep Face Recognition". *CVPR*, 2019.
2. Guo, J.; Deng, J.; Lattas, A.; Zafeiriou, S. "Sample and Computation Redistribution for Efficient Face Detection" (SCRFD). *ICLR*, 2022.
3. Zhang, Y. et al. "ByteTrack: Multi-Object Tracking by Associating Every Detection Box". *ECCV*, 2022.
4. Redmon, J.; Divvala, S.; Girshick, R.; Farhadi, A. "You Only Look Once: Unified, Real-Time Object Detection". *CVPR*, 2016.
5. Desplanques, B.; Thienpondt, J.; Demuynck, K. "ECAPA-TDNN: Emphasized Channel Attention, Propagation and Aggregation in TDNN Based Speaker Verification". *Interspeech*, 2020.
6. Kim, J.; Kong, J.; Son, J. "Conditional Variational Autoencoder with Adversarial Learning for End-to-End Text-to-Speech" (VITS). *ICML*, 2021.
7. Povey, D. et al. "The Kaldi Speech Recognition Toolkit". *IEEE ASRU*, 2011.
8. ONVIF. *Profile S Specification; PTZ Service Specification*. onvif.org.
9. IETF RFC 7826. *Real-Time Streaming Protocol Version 2.0*, 2016 (RTSP 1.0: RFC 2326, 1998).
10. IETF RFC 8446. *The Transport Layer Security (TLS) Protocol Version 1.3*, 2018.
11. IETF RFC 8555. *Automatic Certificate Management Environment (ACME)*, 2019.
12. IETF RFC 3164. *The BSD syslog Protocol*, 2001.
13. IETF RFC 5905. *Network Time Protocol Version 4*, 2010.
14. OASIS. *MQTT Version 5.0*, OASIS Standard, 2019.
15. IEEE Std 802.3bt-2018. *Physical Layer and Management Parameters for Power over Ethernet over 4 pairs*.
16. IETF RFC 9106. *Argon2 Memory-Hard Function for Password Hashing and Proof-of-Work Applications*, 2021.
17. OWASP. *CycloneDX Bill of Materials Specification 1.6*, 2024.
18. Regulation (EU) 2016/679 (GDPR), in particular Art. 2 (household exemption) and Art. 9 (biometric data).

APPENDIX C INDEX

Page numbers refer to the body of the document (chapters, not appendices).

A

Access rules, 5, 7, 17, 20, 22, 29, 35–36
ACME, 32
Adaptive learning, 7, 13, 24, 26, 29, 31, 37
AEC, 15
Alloy / Promtail, 7
Anubis, 6–7, 33–34
API (REST), 6–7, 9, 13, 16, 19, 22, 28, 30, 33, 36–37
ArcFace, 7, 9, 12–14, 29
Argon2id, 19
Authorization window, 5, 19–20
AVX2, 33

B

Backup, 6, 29
BOM, 14
ByteTrack, 9–10, 14, 29, 35

C

CA (certificate authority), 32
certbot, 32
CH340, 18, 33
Cooldown, 17–18, 27
CSP, 6
CSRF, 19, 29
CTS / DTR, 18

D

Dead zone, 10, 26
Deduplication, 13, 26
DNS-01, 32, 34
Door sensor, 7, 17–18, 27, 35

E

ECAPA-TDNN, 7, 16, 20, 24
Embedding, 7–8, 12–16, 20, 24
Event log, 24–25, 35

F

Face search, 8, 13–14, 20, 27, 29–30, 37
fail2ban, 34
FFmpeg, 8, 10, 16, 37
Fixed camera, 11, 26, 36
FT232R / FT232RL, 18, 33

G

Gallery, 7, 12–13, 16, 20, 29
GDPR, 8, 27, 31, 36–37
Grafana, 37

H

H.264 / H.265, 10, 29
Half-duplex, 15
Home Assistant, 5, 7, 25, 30
HSTS, 6

I

Identification by vote, 7, 9–10, 12, 26, 29, 31, 35
InsightFace, 7, 12, 20, 24
IoU, 10

IR, 10, 12, 31, 36

J

jarvis-api, 6, 29
jarvis-cert, 29, 32–33
jarvis-core, 6, 25, 29
journald, 7, 25, 29

K

Kaldi, 8, 15

L

Laplacian (variance of the), 12
LCUS-4, 18
LED indicators, 5, 7–9, 15, 17–18, 21, 27, 37–38
Let's Encrypt, 32

M

MJPEG, 9, 19, 22
MQTT, 7, 30, 37

N

nginx, 6–7, 19, 32–34, 37
node_exporter, 7
Novoform Novomatic 200, 31, 36

O

ONNX / ONNX Runtime, 8, 12, 15–16, 33, 35–36
ONVIF, 6–8, 10–11, 26
OpenVINO, 8, 10, 20, 24, 29, 33
openWakeWord, 7, 15, 20, 24, 29

P

Piper, 7, 16, 20, 24, 29, 37
Porcupine, 7, 15, 27
Preset, 11, 19, 26
Prometheus, 7
Proportional controller, 10
PTZ, 5, 7–12, 19, 29, 35, 37
Pull-up, 18
Pulse, 5, 8–9, 17–19, 21, 23, 27, 29, 31, 35, 38

R

Reed switch, 18
Relay, 5–9, 18, 25, 29, 33, 38
ReSpeaker, 15, 33
Restricted grammar, 15, 27, 29, 37
Retention, 8, 14, 24, 27, 36–37
Retroactive identification, 13, 20, 29–30
RTSP, 6–10, 37

S

SBOM, 29
Schedule (time slot), 29, 35
SCRFD, 7, 9, 12, 29
Session / login, 6, 12, 19, 22, 24–25, 27, 29, 33
Sighting, 5, 7–9, 12–14, 17, 19–20, 22–25, 27, 29–30
Similarity (cosine), 12
Similarity threshold, 12–13, 15–16, 26–27
Speaker verification, 7, 16, 27, 31
SQLite (WAL), 6–7, 9, 13, 24

syslog, 7

systemd, 6–7, 20, 29–30, 32–33

T

TLS / HTTPS, 6, 20, 24, 32–34

tmpfs, 9, 24

Track, 8, 10–14, 19, 26, 35

TTS, 7, 9, 15–16, 21, 27

U

Ubuntu Server, 6, 30

udev, 18

Unix socket, 6–8, 28–29, 33

Unknown cluster, 5, 9, 13–14, 17, 20, 22, 24–26, 29–31

V

VAAPI, 10, 26, 33

VITS, 16

Vosk, 9, 15, 20, 24, 27, 29, 37

W

Wake word, 5, 7, 15, 27, 29, 37

Watchlist, 5, 13–14, 17, 20, 22, 24–25, 27, 29

Webhook, 5, 7–8, 25, 28–29

Y

YOLO, 9–10, 20, 24, 29, 35